Emotion Based Quranic Recommendation System

By

Aneeqa Mosham Idrees

22-ARID-608

Noor-ul-Huda Azhar

22-ARID-864

Supervisor

Ms. Iram Rubab

University Institute of Information Technology
PMAS-Arid Agriculture University Rawalpindi
Pakistan-2026

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software documentation and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

---------------------------	---------------------------
Aneeqa Mosham Idress	Noor ul Huda Azhar

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “Emotion Based Quranic Recommendation System” was developed by “**Aneeqa Mosham Idress, 22-Arid-608”**, “**Noor ul Huda Azhar, 22-Arid-864”** under the supervision of “**Mam Iram Rubab**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Mam Iram Rubab
Supervisor

(Dr. Rehan Mehmood Yousaf)
Examiner-I

(Asad ul Haq Hashmi)
Examiner-II

(Prof. Dr. Yaser Hafeez)
Director UIIT

Executive Summary

In today world, people often face emotional stress, anxiety, sadness, and mental pressure, but they do not always have easy access to personal guidance and emotional support. Traditional support systems depend strongly on human interaction, which can be time-consuming, inconsistent, and not always available. Many people also feel hesitant to share their true emotions openly. Existing digital systems usually provide general motivational content without understanding the users real emotional state, and they do not provide personalized spiritual guidance based on emotions.

To overcome these problems, the Emotion-Based Quranic Guidance System is developed. This system helps users by understanding their emotions through voice input and then providing Quranic verses, translations, and motivational guidance according to their emotional condition. It works as a standalone intelligent system that does not require constant human supervision. The system analyzes the text, emoji, voice of the user, detects emotional patterns, and then suggests relevant Quranic teachings that bring comfort, hope, and spiritual strength.

This system uses modern machine learning and deep learning techniques to process text, emoji, voice data and detect emotions accurately. It stores user interactions and emotional history for future improvement of personalized recommendations. Users can easily access the system through a simple interface. The system is designed to support individuals looking for emotional healing, inner peace, and motivation through Islamic teachings, making it a reliable and secure digital spiritual companion.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task. We are greatly indebted to our project supervisor “Mam Iram Rubab” for personal supervision, advice, valuable guidance and completion of this project. We are deeply indebted to her for encouragement and continual help during this work.We are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

----------------------------- ---------------------------

Aneeqa Mosham Idrees Noor ul Huda Azhar

Abbreviations

BERT	Bidirectional Encoder Representations from Transformers
DistilRoBERTa	HuggingFace transformer model for English emotion classification
Keyword Fallback (Urdu/Arabic)	Domain-specific keyword matching for Urdu and Arabic emotion detection.
RoBERTa	Robustly Optimized BERT Pretraining Approach
SpeechRecognition	Python library for converting speech audio to text via Google Web Speech API
LSTM	Long Short-Term Memory
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
SRS	Software Requirement Specification
SDLC	Software Development Life Cycle
ORM	Object Relational Mapping
MVC	Model View Controller
IDE	Integrated Development Environment
SSD	Solid State Drive
PWA	Progressive Web Application

Table of Contents

List of Figures

List of Tables

Chapter 1: Introduction

The EBQ Recommendation System addresses the emotional struggles of people (sadness, stress, depression) by recommending Quranic verses that bring peace and guidance. Instead of manually searching, the system automatically detects the users emotional state and suggests relevant Ayaat. It can also be used in workplaces to motivate teams with Quranic reminders.

1.1. Brief

The Emotion-Based Quranic Recommendation (EBQR) System is a web application that helps users connect with Allah and the Holy Quran based on their emotional state. The system detects emotions from text, voice, or emoji input using AI and NLP techniques and then recommends relevant Quranic verses, translations, and recitations. Its main goal is to provide peace, comfort, and spiritual guidance by bringing users closer to the Quran through technology.

1.1.1 Introduction:

The Emotion-Based Quranic Recommendation (EBQR) System is a web application made to help people connect with Allah and the Holy Quran in an emotional and peaceful way. In today world, many people face stress, sadness, and anxiety. This project helps users find peace and comfort by showing Quranic verses based on their current emotions.

The system can detect emotions from text, voice, or emoji input. It uses Artificial Intelligence (AI) and Natural Language Processing (NLP) to understand the user mood, such as happiness, sadness, fear, or anger. After detecting the emotion, the system recommends related Quranic verses, translations, and recitations. It can also show motivational Islamic videos to help improve the user mood.

The system uses the DistilRoBERTa transformer model (j-hartmann/emotion-english-distilroberta-base) for English emotion detection, and a domain-specific keyword fallback for Urdu and Arabic inputs. Voice input is handled via the SpeechRecognition library with the Google Web Speech API. All Quranic data, including verses and translations, are taken from dataset.

The main purpose of this project is to use technology to bring people closer to the Quran and help them find peace, guidance, and comfort through the words of Allah.

1.1.2 Outcome

The proposed Emotion-Based Quranic Recommendation (EBQR) System is expected to provide emotional, spiritual, and psychological benefits to users. It will help people connect with Allah and the Holy Quran in a more personal way by recommending Quranic verses according to their emotional state. The system will detect emotions from text, voice, and emoji input and then suggest relevant Quranic Ayat, translations, and recitations. This will help users find peace, reduce stress, and feel spiritually uplifted. The system will serve as a source of comfort and guidance, especially for those experiencing sadness, fear, or anxiety.

1.1.3 Highlights of Discussions in Various Chapters

A very brief introduction about the upcoming chapters:

- **Chapter 2 Problem Statement:**

  This chapter explains that existing Quranic apps lack emotion-based personalization and require users to manually search for verses. It highlights the need for an AI system that understands user emotions and provides relevant Quranic guidance.

- **Chapter 3 Requirement Analysis:**

  This chapter describes the functional requirements such as emotion detection and Quranic recommendations, along with non-functional requirements like performance, security, and usability.

- **Chapter 4 System Design:**

  Describes the overall system architecture, including frontend interface, backend logic, AI emotion detection models, database structure, and API integration for Quranic content.

- **Chapter 5 Implementation:**

  Details the implementation of emotion detection using DistilRoBERTa for English text, keyword-based fallback for Urdu and Arabic, and Speech Recognition with Google Web Speech API for voice input, along with the development of the web application and recommendation module.

- **Chapter 6 Testing and Evaluation:**

  Covers testing strategies used to evaluate emotion detection accuracy, system performance, usability, and the effectiveness of Quranic recommendations.

- **Chapter 7 Conclusion and Future Work:**

  Summarizes the achievements of the EBQR System, discusses its benefits in providing emotional and spiritual support, and suggests future improvements such as mobile app development and advanced personalization.

1.1.4 Tools and Technologies

This table include the tools and technologies we can use in our project:

Table 1. 1: Tools and Technologies

	Tools	Version
Tools And Technologies	**Tools**	**Version**
	VS Code (latest)	1.90+
	MySQL	8.0 +
	Draw.io	28.2.9
	MS Word	2013+
	MS Power Point	2013+
	Python	3.12
	Technology	**Version**
	Html	5
	CSS	3
	JavaScript	2021

1.2. Relevance to Course Modules

The Emotion-Based Quranic Recommendation System project is closely related to several courses studied during our degree program:

- **Artificial Intelligence and Machine Learning:**

  This course helped us apply Natural Language Processing (NLP) and the DistilRoBERTa transformer model (BERT-based) and Speech Recognition library for detecting user emotions from text, voice, and emoji inputs.

- **Database Management System (DBMS):**

  Knowledge from this course was used to design and manage databases for storing Quranic verses, translations, detected emotions, and user history efficiently.

- **Software Engineering:**

  Concepts from this course guided us in designing diagrams such as the Class Diagram and Use Case Diagram. We also followed the Agile Software Development Life Cycle (SDLC) for modular development.

- **Information and Communication Technology (ICT):**

This course enabled us to develop an interactive website using HTML, CSS, and JavaScript to ensure a smooth and user-friendly experience.

- **Islamic Studies:**

The foundation of this project lies in Islamic values, promoting spiritual well-being and emotional balance by recommending Quranic verses to guide users toward peace and closeness to Allah (SWT).

1.3. Project Background

This project is designed to engage people with Allah (SWT) and the Holy Quran in a more emotional and personal way. It helps users connect spiritually by offering Quranic verses that match their feelings and emotional states such as sadness, happiness, fear, or stress. The system analyzes user input through text, voice, or emoji and recommends relevant verses and translations to provide guidance, motivation, and inner peace. By combining modern Artificial Intelligence techniques with Islamic values, the project encourages users to seek closeness to Allah and reflect upon the teachings of the Quran in their daily lives.

This project also focuses on reducing stress, depression, and anxiety by offering emotional and spiritual comfort through the words of Allah. It aims to bring calmness and positivity to user's heart by reminding them of Allah mercy, hope, and guidance. Through this approach, the system serves as both a spiritual companion and a source of mental well-being for users.

Furthermore, the project ensures authenticity by using verified and trusted Quranic sources through official APIs such as Al-Quran Cloud editions, making it a reliable and authentic Quranic recommendation website for all users.

1.4. Literature Review

There are several existing applications and research works related to the Quran and emotion detection, but none of them combine both domains effectively into a single intelligent system.

Existing Islamic applications such as Ayat**,** Quran Companion**,** Muslim Pro, and Tanzil Quran provide access to the Quran in the form of Arabic text, translations, tafseer, bookmarking features, and audio recitations by different Qaris. These applications are useful for reading and listening purposes; however, they function mainly as digital Quran libraries. They do not incorporate artificial intelligence techniques or emotion-based personalization. Users are required to manually search for verses using keywords or by selecting specific Surah's. As a result, there is no mechanism to understand the emotional state of the user or to recommend verses dynamically based on feelings such as sadness, anxiety, or gratitude.

Similarly, some Islamic counseling platforms and motivational Islamic quote applications provide categorized verses under themes like patience, gratitude, or hope. However, these systems are static and rule-based. They rely on predefined categories rather than real-time emotional analysis. They do not analyze user-written text, voice input, or emoji input to determine emotional context.

On the other hand, general sentiment analysis systems and emotion detection models have been widely developed in the field of Machine Learning(ML). For example, many research projects use Twitter datasets to detect emotions such as happiness, sadness, anger, fear, and surprise. Pre-trained transformer models available on platforms like Hugging Face, BERT-based classifiers, and LSTM-based deep learning models are capable of accurately identifying emotions from English text. These systems are used in areas such as customer feedback analysis, mental health monitoring, and social media analytics. Moreover, some mental health chatbots provide motivational responses based on detected emotions, but they rely on psychological coping strategies rather than religious or Quranic guidance. They do not integrate structured Islamic knowledge or authenticated Quranic content into their recommendation systems.

Therefore, a research gap clearly exists. While Quran applications provide authentic Islamic content without emotional intelligence, and sentiment analysis systems provide emotion detection without spiritual context, there is no integrated system that combines both. The proposed Smart Emotion-Based Quran Recommendation System aims to bridge this gap by detecting the user's emotional state through text, voice, or emoji input and recommending relevant Quranic Ayat accordingly. This integration introduces personalization, emotional awareness, and spiritual guidance within a single intelligent platform.

1.5. Analysis from Literature Review

In comparison with the existing systems discussed in the literature review, our project introduces a more intelligent and personalized approach by combining emotion detection with Quranic recommendations. Unlike current Islamic apps such as Ayat or Quran Companion, which require users to manually search for verses, our system automatically analyzes the users emotional state through text, voice, or emoji inputs. It then recommends Quranic verses that are relevant to the detected emotion, providing comfort, guidance, and motivation.

Compared to general sentiment analysis systems, our project goes beyond simply detecting emotions and it uses those emotions to deliver meaningful spiritual support. By integrating Machine Learning(ML), deep learning models like BERT and Speech Recognition, and authentic Quranic datasets, our system bridges the gap between technology and spirituality, offering a unique, emotion-aware Islamic recommendation platform.

1.6. Methodology and Software Lifecycle for This Project

The Agile method is used for the “Emotion-Based Quranic Recommendation System” project. This method is flexible and allows the project to be developed step by step. It helps the team get feedback from users and make changes during development. Agile also improves teamwork and makes it easier to create a system that is accurate, easy to use, and helps people connect with the Quran in an emotional way.

1.6.1 Agile Methodology:

- **Iterative Development:**

  Agile divides the project into small parts like text emotion detection, voice input, emoji input, and Quranic recommendation. Each part is developed and tested one by one. This helps fix problems early and make improvements at every step.

- **Flexibility:**

  Agile is very flexible. If new ideas or changes are needed, they can be added easily. For example, if users want better translations or more emotion options, they can be added without restarting the whole project.

- **Continuous Feedback Integration:**

  Agile focuses on getting feedback from users, teachers, and mentors throughout development. This helps the team improve the systems accuracy, design, and performance. By using feedback, the final system becomes more useful and provides real emotional comfort and Quranic guidance to users.

1.6.2 Object-Oriented Approach:

- **Clear Structure:**

  The Object-Oriented approach helps organize the system into clear classes such as User, Emotion, Recommendation, and Database, making the code easy to understand and manage.

- **Code Reusability:**

  It allows the reuse of existing code for similar features, which saves time and effort during development.

- **Modularity:**

  Each class works as a separate module, so changes in one part do not affect the whole system.

- **Easy Debugging:**

  Errors can be found and fixed easily because each part of the system is independent and well-structured.

- **Efficient and Adaptable System:**

  Combining Object-Oriented design with the Agile method keeps the project flexible, efficient, and easy to improve, ensuring a user-friendly and high-quality final product.

Chapter 2: Problem Definition

This chapter discusses the precise problem to be solved. It should extend to include the outcome This chapter describes the problem definition of our proposed system. This chapter gives a better understanding of the project. Chapter 2 is based on the brief problem definition of the project which gives a detailed insight of this project and a better understanding. This chapter will explain the purpose of our system. This chapter explains why we are going to make this system and how it will be different from other current systems and how it will be useful by the end users. What types of problems will this project solve?

2.1. Problem Statement

The existing systems are limited because they mainly rely on text input and recognize only a small range of emotions. They lack voice and emoji input capabilities and offer limited language support**,** making it difficult to understand the diverse emotional expressions of users.

In today fast-paced world, many individuals experience sadness, anxiety, and depression, creating a strong need for a system that not only detects emotions but also provides spiritual and emotional comfort.

Our proposed system fills this gap by using Artificial Intelligence (AI) to detect a user emotions from text, voice, and emoji inputs and then recommend relevant Quranic Ayat that offer peace, hope, and guidance. This project not only applies advanced AI and Machine Learning (ML) techniques but also helps in strengthening our personal connection with the Holy Quran**,** promoting emotional healing through divine words.

2.2. Deliverables and Development Requirements

The Following are the main deliverables and development need that the project will produce:

2.2.1 List of Deliverables

- **Web Application:**

  A complete and functional website that detects user emotions through text, voice, and emoji inputs and recommends relevant Quranic Ayat with translations, recitations, and tafseer.

- **Dashboard:**

  An interactive dashboard displaying detected emotions, user history, and recommended verses along with motivational content and emotional statistics.

- **User Authentication:**

  A Sign-up and Login system that stores user data securely for personalized recommendations and feedback monitoring.

2.2.2 Development Requirements:

In this we will discuss about the software requirements and development tools:

- **Software Requirements:**

  Frontend: HTML5, CSS3, JavaScript, React.
  Backend: Python (Flask).
  AI / ML Libraries: HuggingFace Transformers (DistilRoBERTa — j-hartmann/emotion-english-distilroberta-base), Speech Recognition, Pandas, NumPy.
  Database: MySQL .
  APIs: Google Speech-to-Text API (voice input), YouTube API (motivational Islamic videos).
  Dataset: Quranic Data

- **Development Tools:**

  Visual Studio Code (coding and development).
  Draw.io (for diagrams such as use case, and UI mockups).
  MS Word and PowerPoint (for documentation and presentation).

2.3. Current System

There are some similar projects already available in the market but they have some major drawbacks this project removes those deficiencies and overcomes those drawbacks.

2.3.1 Ayat App

The Ayat App is a popular Islamic mobile application developed to help users read, listen, and understand the Holy Quran. It provides complete Quranic text along with translations and tafseer in multiple languages. Users can listen to recitations from famous Qaris and explore tafseer for a deeper understanding of each verse.

However, despite its usefulness, the Ayat App does not analyze user emotions or provide personalized Quranic recommendations. The user must manually search for verses related to a specific topic or emotion such as sadness, happiness, or peace. Therefore, while it is beneficial for Quranic study, it lacks emotional interaction and intelligent guidance.

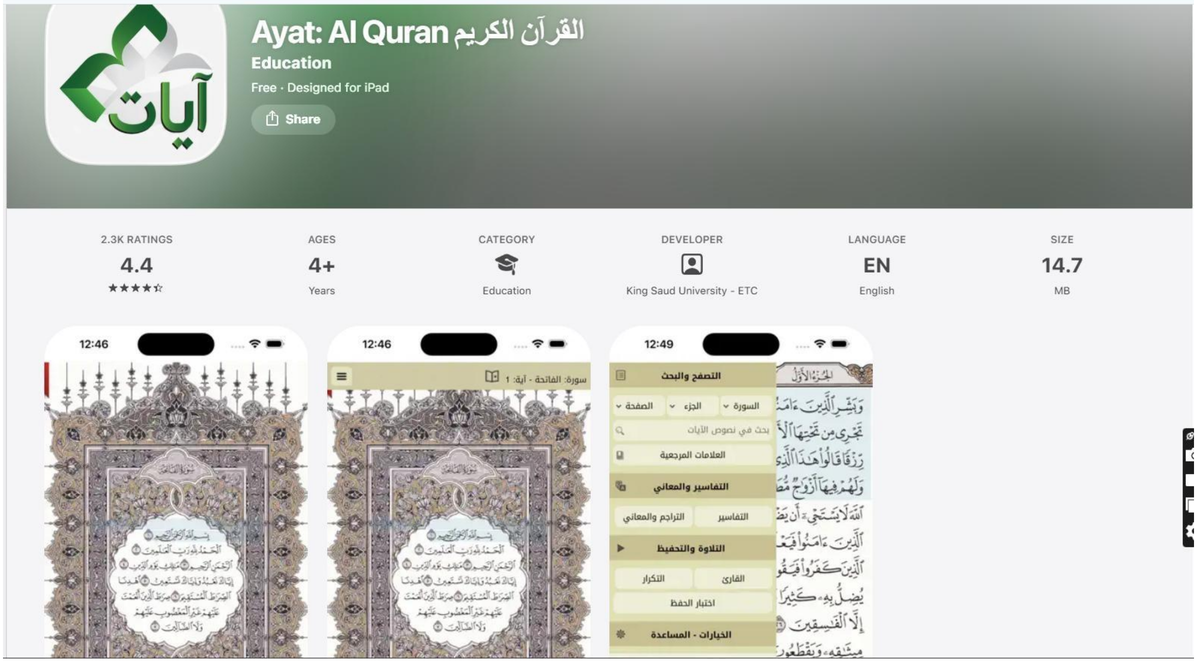

Figure 2. 1:Ayat App

Table 2. 1:Ayat App comparison with Ayat EBQ Recommendation System

Ayat App	EBQ Recommendation System
Users manually search for verses.	Automatically recommends verses based on detected emotions.
Only supports text-based reading.	Accepts text, voice, and emoji inputs to detect user emotions
Basic database and UI for reading and listening.	Uses AI, NLP, a dual-model pipeline (j-hartmann -DistilRoBERTa for English; XLM-RoBERTa + keyword scorer for Urdu/Arabic), and langdetect for automatic language identification, for personalized Quranic guidance.

2.3.2 Quranic Sentiments System

The Quranic Sentiments System is an AI-based research project that applies sentiment analysis and natural language processing (NLP) techniques to the Holy Quran. Its main goal is to analyze Quranic verses and identify their emotional or sentiment orientation such as positive, negative, or neutral. This helps researchers and developers understand how different verses convey emotional tones or thematic meanings.

However, the Quranic Sentiments System focuses mainly on analyzing the text of the Quran itself, not on understanding the user's emotions. It provides sentiment scores for Quranic text but does not give personalized recommendations or emotional comfort to users.

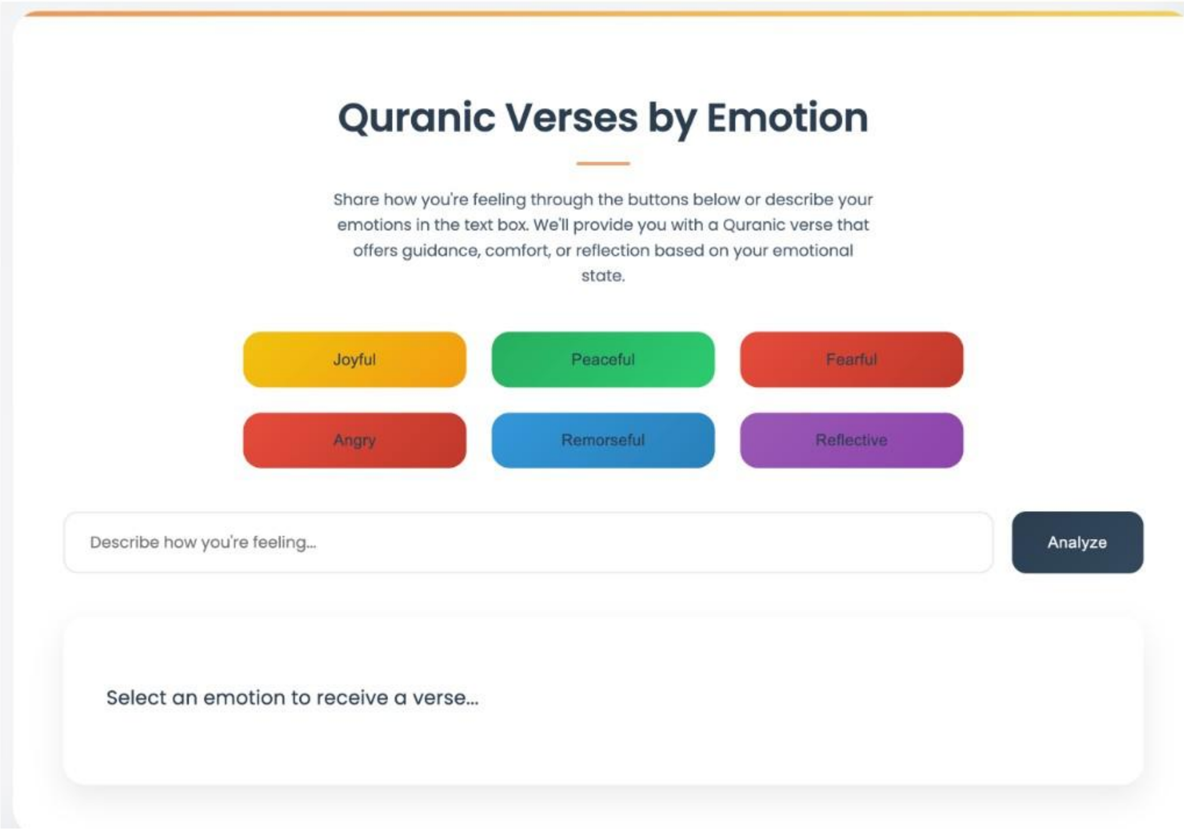

Figure 2. 2:Quranic Sentiment System

Table 2. 2:Quranic sentiments System comparison with EBQ Recommendation System

Quranic Sentiment System	**EBQ Recommendation System**
Analyzes emotional tones within Quranic text using sentiment analysis.	Detects user emotions (text, voice, emoji) and recommends Quranic verses for guidance.
Focuses on finding positive or negative sentiments in Quranic verses.	Focuses on user's emotional state and provides personalized Quranic recommendations.
Classifies verses by sentiment polarity (positive, neutral, negative).	Provides related translations, and recitations to comfort users emotionally.

Chapter 3: Requirement Analysis

This chapter describes the functional and non-functional requirements of our proposed system. Chapter 3 is based on the functional and non-functional requirements of the project which gives a detailed insight of this project and a better understanding. In this chapter we will briefly explain the actor's description. We will explain our proposed system actors and also describe their functional. After actor's description this chapter briefly describes the use cases of our system. In each use case we describe the description of the use case, trigger, preconditions and post conditions, normal flow of the use case also explains the alternative flows and exceptions. This chapter gives a better understanding of modules of project and also their functionalities.

3.1. Use Cases

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a use case analysis. Its purpose is to present a graphical overview of the functionality provided by an application in terms of Actors, their goals, and dependencies. The main use case diagram of our system is shown below:

3.1.1 Use Cases Diagram

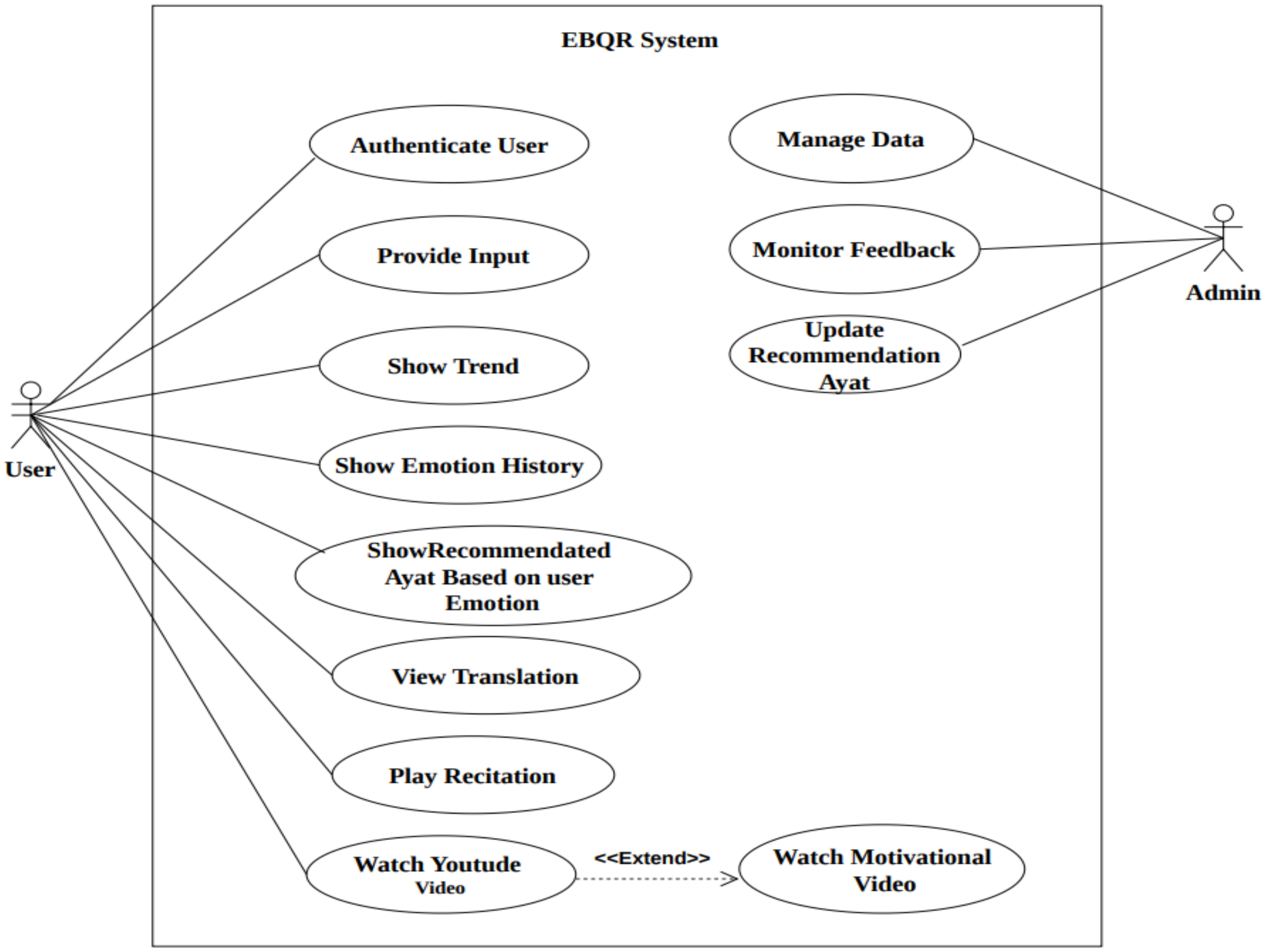

Figure 3. 1: Use Case Diagram

3.1.2 Actors Description

A very Brief introdunction of the Roles and relationships to the System:

Table 3. 1:Actors Description

Actors	Job Role	Description
User	Login, provide emotional input (text/voice/emoji), view emotion trends, view emotion history, receive Quranic recommendations, view translations, listen to recitation, and watch motivational videos.	The user is the main actor of the system. They interact with the platform to share their emotional state and receive personalized Quranic verses, translations, recitations, and motivational content for spiritual guidance, peace, and emotional support. The user can also track their emotional trends and history through the system dashboard.
Admin	Manage system data, monitor user feedback, update Quranic verse recommendations, and control system content.	The Admin role is planned as a future enhancement. In the current system version, administrative functions (feedback monitoring, verse dataset management, user account oversight) are queued for a future development sprint. All user-facing features are fully implemented.

3.1.3 Use Case Description

Table 3. 2:Authenticate User

Use Case ID:	UC-1
Use Case Name:	Authenticate User
Actors:	User ,Admin

Description:	Allows the user to log in to the system.
Trigger:	User opens the login page.
Preconditions:	User has access to the system.
Post conditions:	1. User account is successfully created.
Normal Flow:	1. User selects “Sign Up”. 2. User enters name, email, and password. 3. System validates the input. 4. System creates a new user account.
Alternative Flows:	User uses social login if available.
Exceptions:	If email already exists, user is notified.
Includes:	1. Validate user inputs 2. Save user data
Special Requirements:	Secure authentication
Assumptions:	1. Users have valid login credentials. 2. The authentication server is active. 3. Internet connection is available.
Notes and Issues:	1. Weak passwords may cause security risks. 2. Users may forget passwords. 3. Multiple failed login attempts may lock the account.

Table 3. 3:Provide Input

Use Case ID:	UC-2
Use Case Name:	Provide Input
Actors:	User

Description:	User provides emotional input in text, voice, or emoji form
Trigger:	User selects an input option.
Preconditions:	User is logged in.
Post conditions:	Input is stored successfully.
Normal Flow:	1. User selects input type. 2. User enters or records input. 3. System saves input.
Alternative Flows:	If the input is unclear or invalid, the system asks the user to try again.
Exceptions:	1. System unable to connect to Quran API → Show error message "Connection Error, Please Try Again." 2. Voice input too noisy → Prompt user to record again.
Includes:	1. Voice recording 2. Text processing 3. Emoji
Special Requirements:	1. System should respond within 3–5 seconds after input. 2. The interface should be simple and accessible on both web and mobile devices. 3. Support for multiple languages (English, Urdu).
Assumptions:	1. User understands one of the supported languages (English, Urdu,) and has stable internet access. 2. Users can write text, record voice, or select emoji. 3. Input devices (keyboard/microphone) are functional.
Notes and Issues:	1. Voice input may be unclear due to noise. 2. Text input may contain spelling errors. 3. Emoji selection may not fully express emotions.

Table 3. 4:Show Trend

Use Case ID:	UC-3
Use Case Name:	Show Trend
Actors:	User
Description:	Shows emotional trends of the user over time.
Trigger:	User clicks "Show Trend"
Preconditions:	User has history data available.
Post conditions:	Trend chart is displayed.
Normal Flow:	1. User selects trend option. 2. System fetches history. 3. System displays graphs.
Alternative Flows:	No data found message.
Exceptions:	Database connection error.
Includes:	Data retrieval
Special Requirements:	Graph visualization library
Assumptions:	1. Emotion history data exists in the database. 2. Visualization modules are properly configured.
Notes and Issues:	Trends may be inaccurate if data is incomplete. Graphs may load slowly with large datasets.

Table 3. 5:Show Emotion History.

Use Case ID:	UC-4
Use Case Name:	Show Emotion History.
Actors:	User
Description:	Displays previously detected emotions and activities.
Trigger:	User clicks history option.
Preconditions:	User has previous data.
Postconditions:	History is displayed.
Normal Flow:	1. User opens history page. 2. System fetches saved data. 3. History is shown.
Alternative Flows:	No history found.
Exceptions:	Database connection failure.
Includes:	1. Fetch records 2. Display list
Special Requirements:	Secure user data access
Assumptions:	1. The system stores past user emotion records. 2. Users allow permission for data storage.
Notes and Issues:	1. Missing or corrupted records may affect results. 2. Privacy risks if data is not secured.

Table 3. 6:Show Recommended Ayat Based on Emotion

Use Case ID:	UC-5
Use Case Name:	Show Recommended Ayat Based on Emotion
Actors:	User
Description:	Displays Quranic verses based on detected emotion.
Trigger:	Emotion is detected.
Preconditions:	Emotion data exists.
Postconditions:	Recommended ayat are displayed.
Normal Flow:	1. System matches emotion with verses. 2. System retrieves ayat data. 3. System shows recommendations.
Alternative Flows:	No recommendations found.
Exceptions:	Quran API failure.
Includes:	1. Emotion mapping 2. Verse fetching
Special Requirements:	Fast recommendation processing.
Assumptions:	1. Emotion-to-Ayat mapping logic is available. 2. Qur’an database is properly indexed.
Notes and Issues:	1. Recommendations may not always fully match the user feeling. 2. API or database failure may stop verse retrieval.

Table 3. 7:View Translation

Use Case ID:	UC-6
Use Case Name:	View Translation
Actors:	User
Description:	Allows user to view translation of Quranic verses.
Trigger:	User clicks translation button.
Preconditions:	Verses are displayed.
Postconditions:	Translation is shown.
Normal Flow:	1. User selects verse. 2. System loads translation. 3. Translation is displayed.
Alternative Flows:	User changes language.
Exceptions:	Translation not available.
Includes:	1. Translation loader 2. Language selection
Special Requirements:	Multiple language support
Assumptions:	1. Multiple language translations exist in the system. 2. Users have selected a preferred language
Notes and Issues:	1. Translation accuracy may vary between languages. 2. Some languages may not be available.

Table 3. 8:Watch YouTube Video

Use Case ID:	UC-7
Use Case Name:	Watch YouTube Video
Actors:	User
Description:	Allows user to watch YouTube videos related to emotions.
Trigger:	User clicks video link.
Preconditions:	Internet connection available.
Postconditions:	Video is played.
Normal Flow:	1. User clicks video. 2. System opens YouTube player. 3. Video starts.
Alternative Flows:	Slow internet connection.
Exceptions:	YouTube not accessible.
Includes:	1. Video embedding 2. Playback controls
Special Requirements:	Stable internet
Assumptions:	1. YouTube API access is enabled. 2. Internet speed supports video streaming.
Notes and Issues:	1. Some videos may be unavailable or restricted. 2. Ads may interrupt user experience.

Table 3. 9:Manage Data

Use Case ID:	UC-9
Use Case Name:	Manage Data
Actors:	Admin
Description:	Admin manages system data.
Trigger:	Admin logs in.
Preconditions:	1. Admin is authenticated. 2. Database connection is active..
Postconditions:	1. Selected data is updated or stored. 2. Changes become available to users
Normal Flow:	1. Admin opens the dashboard. 2. Admin selects "Manage Data." 3. System displays data management options. 4. Admin edits/updates/inserts data. 5. System validates and saves changes.
Alternative Flows:	Admin cancels the update. System rejects invalid data and asks for correction.
Exceptions:	1. Database not reachable. 2. Input format invalid. 3. Duplicate entries detected.
Includes:	1. Update Quranic Ayat 2. Update Emotion-Ayat Mapping 3. Update Translations 4. Update Recitations
Special Requirements:	1. Secure authentication 2. Data validation checks 3. Role-based access control
Assumptions:	1. Admin has full system access rights. 2. Database connection is stable.

Notes and Issues:	1. Unauthorized access may cause data leakage. 2. Data loss may occur if backups are not maintained.

Table 3. 10:Monitor Feedback

Use Case ID:	UC-10
Use Case Name:	Monitor Feedback
Actors:	Admin
Description:	Admin monitors user feedback.
Trigger:	Admin opens feedback page.
Preconditions:	1. Users have submitted feedback. 2. Feedback database is active.
Postconditions:	Feedback is displayed.
Normal Flow:	1. Admin navigates to the feedback section. 2. System displays feedback list. 3. Admin opens a feedback entry. 4. Admin responds or marks it resolved.
Alternative Flows:	Admin filters feedback by category. Admin exports feedback for review.
Exceptions:	No feedback available. Database retrieval error.
Includes:	View-Feedback Respond to Feedback
Special Requirements:	Feedback privacy Secure data storage
Assumptions:	1. Users actively submit feedback. 2. Feedback storage system is active.

Notes and Issues:	1. Unauthorized access may cause data leakage. 2. Data loss may occur if backups are not maintained

Table 3. 11:Recommendation Ayat

Use Case ID:	UC-11
Use Case Name:	Update Recommendation Ayat
Actors:	Admin
Description:	Admin updates Quranic ayat mappings.
Trigger:	Admin selects “Update Ayat Recommendations.”
Preconditions:	1. Admin is logged in. 2. Emotion categories exist in database.
Postconditions:	Updated recommendations are stored and shown to users based on their emotions.
Normal Flow:	1. Admin opens recommendation settings. 2. System displays emotions list. 3. Admin selects an emotion. 4. Admin updates ayat reference and details. 5. System saves the new mapping.
Alternative Flows:	Admin adds multiple ayat for one emotion. Admin deletes an ayat mapping.
Exceptions:	Invalid ayat reference. Missing translation or surah ID.
Includes:	Validate-Ayat-Source Save Recommendation Data
Special Requirements:	Verified Quran API source Data integrity rules

Assumptions:	1. Admin has verified religious sources. 2. System allows dynamic updating of mappings.

3.2. Functional Requirements

Functional requirements describe what the system should do and what actions it must perform. For this project, the following functional requirements are considered:

3.2.1 User Authentication

The system must allow users to create an account by signing up, log in using secure sign-in functionality, and sign out safely when required. It must securely store user credentials and validate them to ensure data protection and privacy.

3.2.2 Input Handling

The system must allow users to create an account by signing up, log in using secure sign-in functionality, and sign out safely when required. It must securely store user credentials and validate them to ensure data protection and privacy.

3.2.3 Emotion Detection

The system must be capable of analyzing text input to detect the user emotional state, processing voice input for emotion recognition, and mapping emojis to their corresponding emotions. It must support the detection of multiple emotional states such as happiness, sadness, stress, anxiety, gratitude, anger, hopefulness, and loneliness.

3.2.4 Recommendation Quranic Ayat

The system must map the detected emotions to relevant Quranic verses and present them in Arabic along with their English and Urdu translations. It must also provide access to audio or video recitations and explanations to help console and emotionally support the user or consoling the user.

3.2.5 Dashboard

The system must display detected emotions on the dashboard and show recommended Quranic verses with translations and optional audio or video content. It must maintain a history of user interactions and visualize emotion trends over time.

3.2.6 Feedback Mechanism

The system must allow users to provide feedback by marking recommendations as helpful or not helpful. It must store this feedback and use it to improve future recommendations.

3.2.7 User Profile Management

The system must allow users to view and update their personal profile information. It must maintain a history of past emotions, preferences, and feedback and use this data to personalize future recommendations.

3.3. Non-Functional Requirements

Non-functional requirements describe the quality and performance features of the system rather than its specific functions. For this project, the following non-functional requirements are considered:

3.3.1 Performance:

The system should detect user emotions and display related Quranic recommendations within 3–5 seconds for text and emoji inputs, and within 7–10 seconds for voice inputs.

3.3.2 Reliability:

The system should provide consistent results with at least 90% uptime and handle multiple user requests without system crashes or loss of data.

3.3.3 Usability:

The web interface should be simple, user-friendly, and responsive, ensuring easy access for users of all ages and backgrounds. The design will use clear buttons, icons, and instructions.

3.3.4 Security:

User data, including emotions, input text, and history, must be stored securely using encrypted connections (HTTPS) and secure database handling techniques.

3.3.5 Accuracy:

The emotion detection model should achieve at least 85% accuracy in identifying emotions from text, voice, and emoji inputs after model training and testing.

3.3.6 Scalability:

The system architecture should allow easy integration of new features such as additional languages, voice datasets, or AI models in the future.

3.3.7 Maintainability:

The codebase should be modular and well-documented, allowing future developers to modify or extend the system easily.

3.3.8 Compatibility:

The web application must be accessible through all major browsers (Chrome, Firefox, Edge) and devices (desktop, tablet, mobile).

3.3.9 Availability:

The system should remain available to users 24/7, except during scheduled maintenance or updates.

3.3.10 Cultural and Ethical Sensitivity:

The system must ensure respectful handling of Quranic content, using only authentic sources and verified translations, without altering or misrepresenting verses.

Chapter 4: Design and Architecture

This chapter describes the Design and Architecture system. Chapter 4 is based on the UML structural diagrams, behavioral diagrams and UML interaction diagrams of the project which gives a detailed insight of this project and a better understanding. This chapter also describes what type of architecture will be used to develop this project. It will explain what are the advantages while using any type of system architecture. In this chapter we will briefly explain design of our system. UML (Unified Modeling Language) diagrams are commonly used in the software development lifecycle and can be utilized both before and after the coding or implementation phase. These diagrams provide a visual representation of various aspects of a project, facilitating better understanding and communication among team members. Before the coding phase, UML diagrams help in the design and planning stages of a project.

4.1. System Architecture

The proposed Emotion-Based Quranic Recommendation System uses a client–server architecture. The system follows a three-tier architecture that includes the presentation layer, application layer, and database layer. This architecture was chosen because it makes the system faster, more secure, and easier to manage.

4.1.1 Presentation Layer:

In this layer we will taught about frontend layer that present Quranic recommendation interface and captures the user emotion input:

- **Web Interface (React.js):**

The user interface is developed using React.js to build a modern, fast, and interactive web application. React helps create reusable components and provides a smooth user experience for users interacting with the system.

- **Voice Recording Interface:**

The system uses the microphone through the browser to allow users to record voice input easily. This helps users express their emotions naturally.

- **Design Tools:**

HTML, CSS, and JavaScript are used along with React to create a clean, responsive, and user-friendly interface.

4.1.2 Application Layer:

In this layer we will taught about backend layer that handle logics, processing request and communicate with emotion models:

- **Backend Language (Python):**

Python is used to develop the backend services that handle audio processing, emotion detection, and Quranic recommendation logic.

- **Machine Learning Models** (**Speech Recognition**):

Pre-trained speech models like Speech Recognition are used to analyze voice data and detect user emotions accurately.

- **Backend Framework (Flask / FastAPI):**

Flask or FastAPI is used to build REST APIs that connect the frontend (React) with the backend logic.

- **Dataset:**

Quranic Data are used to fetch authentic Quranic verses, translations.

4.1.3. Database Layer:

In this layer we will taught about the database layer that use to store and manage user data and records:

- **Database System (MySQL):**

A structured database is used to store user data, emotion results, session history, and recommendations.

- **Cloud Storage:**

Cloud services can be used to store uploaded audio files securely.

- **Version Control (GitHub):**

Git and GitHub are used for source code management and team collaboration.

- **Technology Compatibility:**

All selected technologies work well together to ensure smooth data flow between the user interface, backend processing, and database.

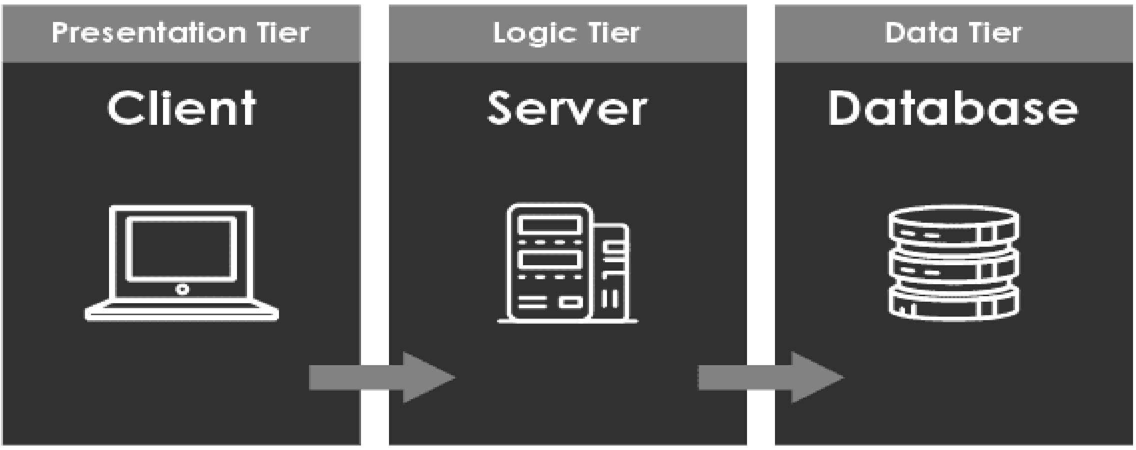

Figure 4. 1:Three Tier System Architecture

4.2. System Design

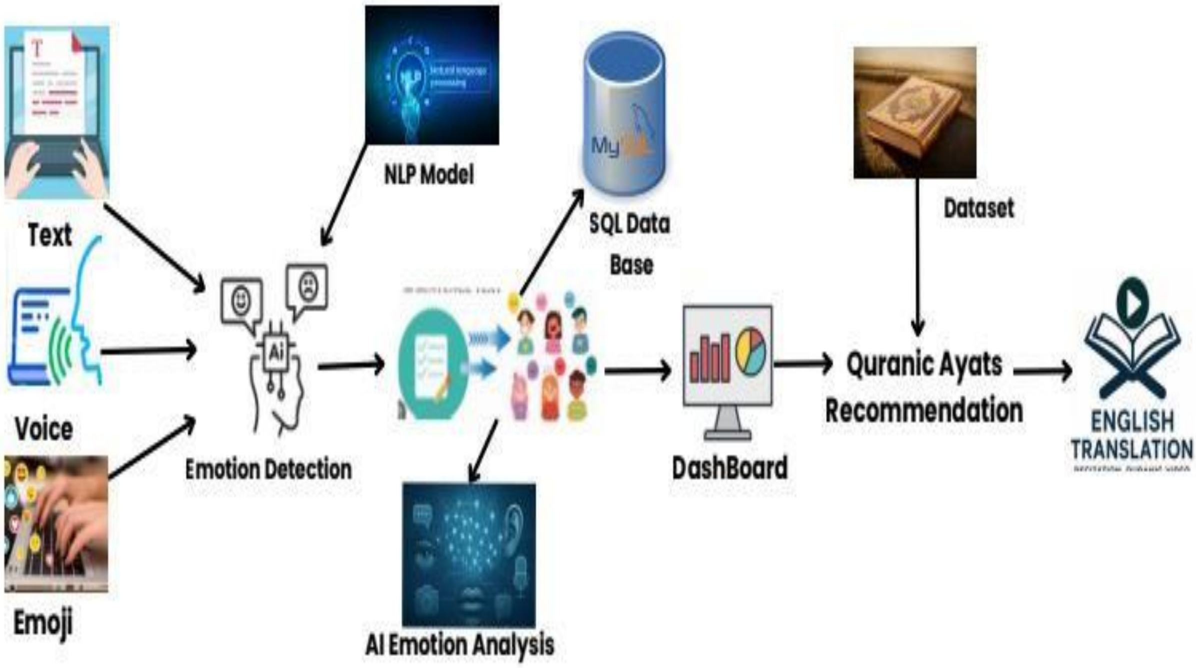

Figure 4. 2:Block Diagram

4.3. UML Structural Diagrams

4.3.1 Class Diagram

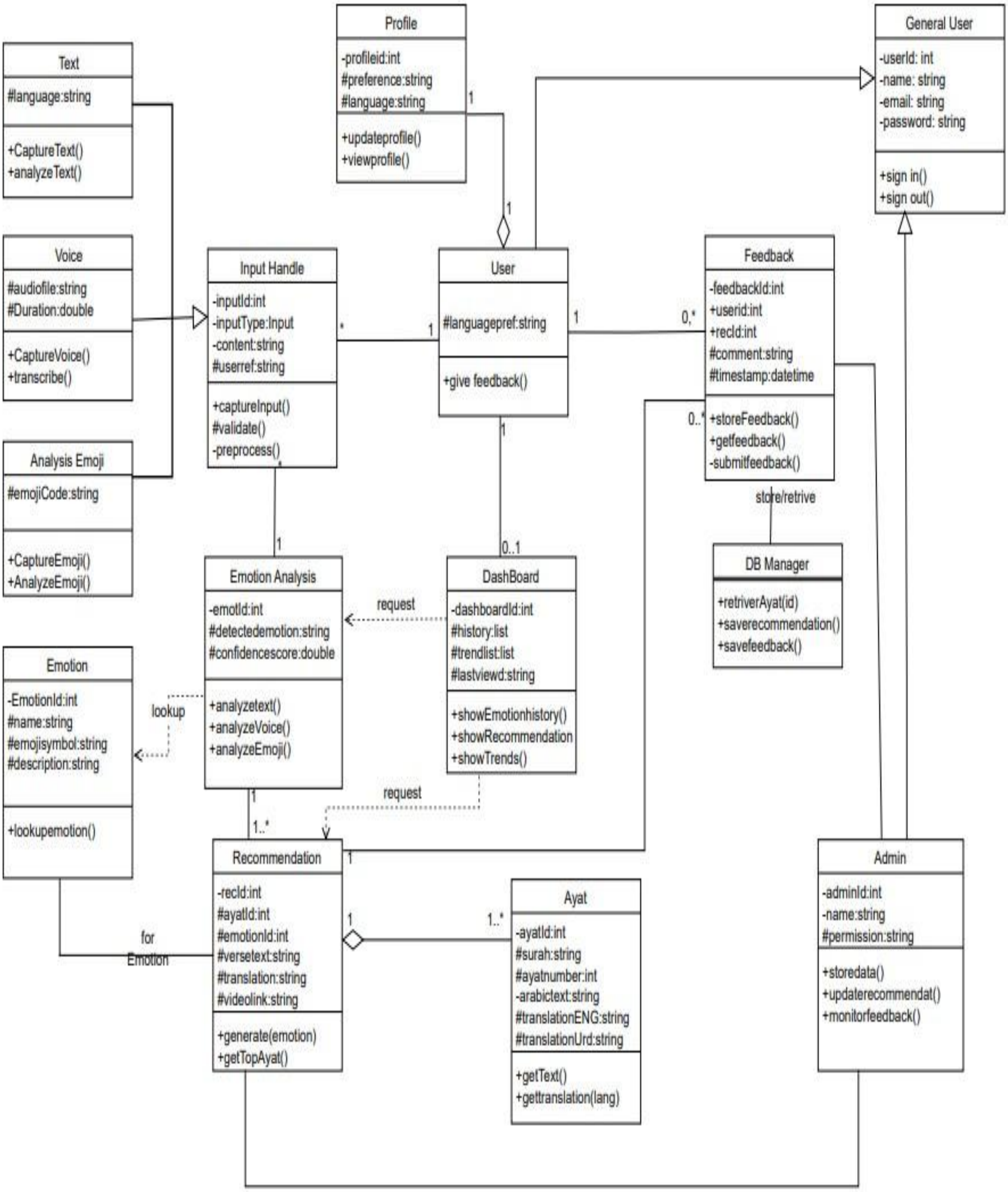

Figure 4. 3:Class Diagram

4.3.2 Component Diagram

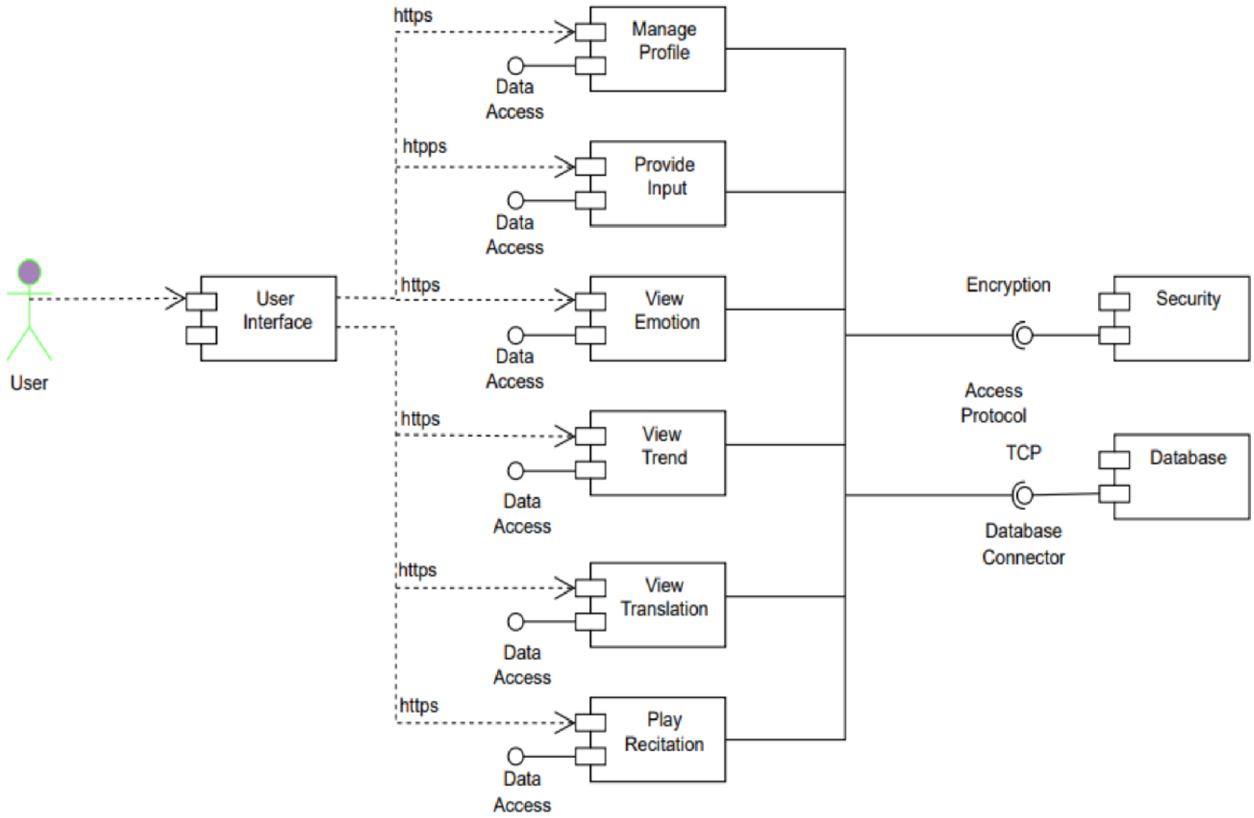

Figure 4. 4:User Component Diagram

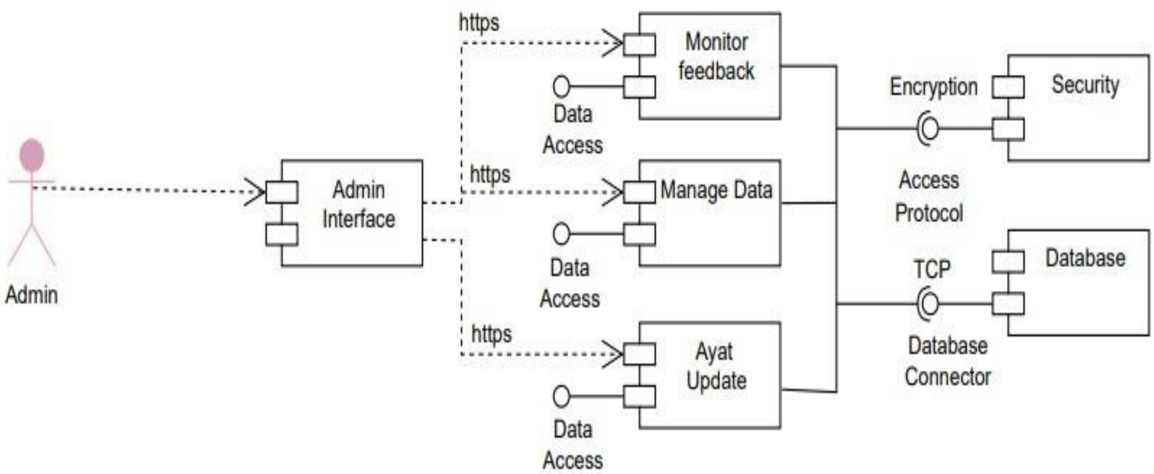

Figure 4. 5:Admin Component Diagram

4.3.3 Package Diagram

Figure 4. 6:Package Diagram

4.3.4 Deployment Diagram

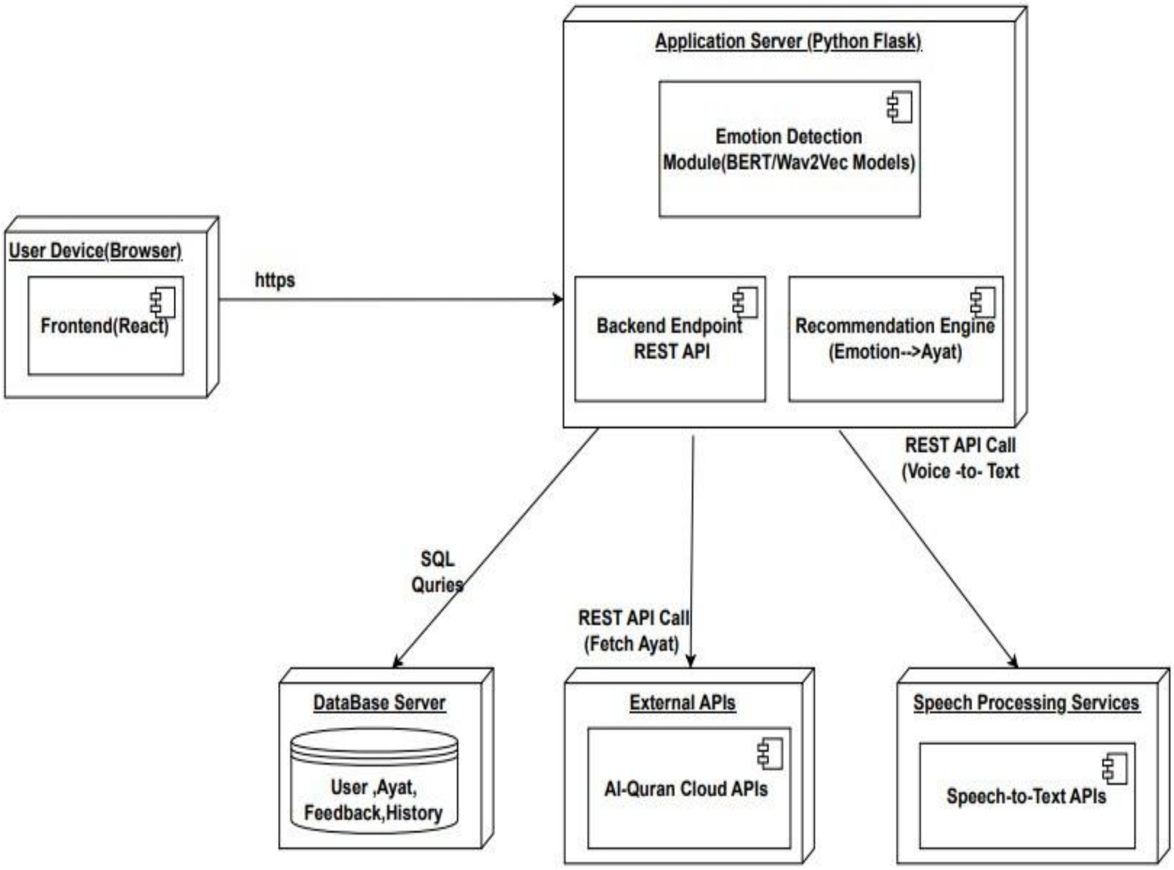

Figure 4. 7:Deployment Diagram

4.4. UML Behavioral Diagrams

4.4.1 Activity Diagrams

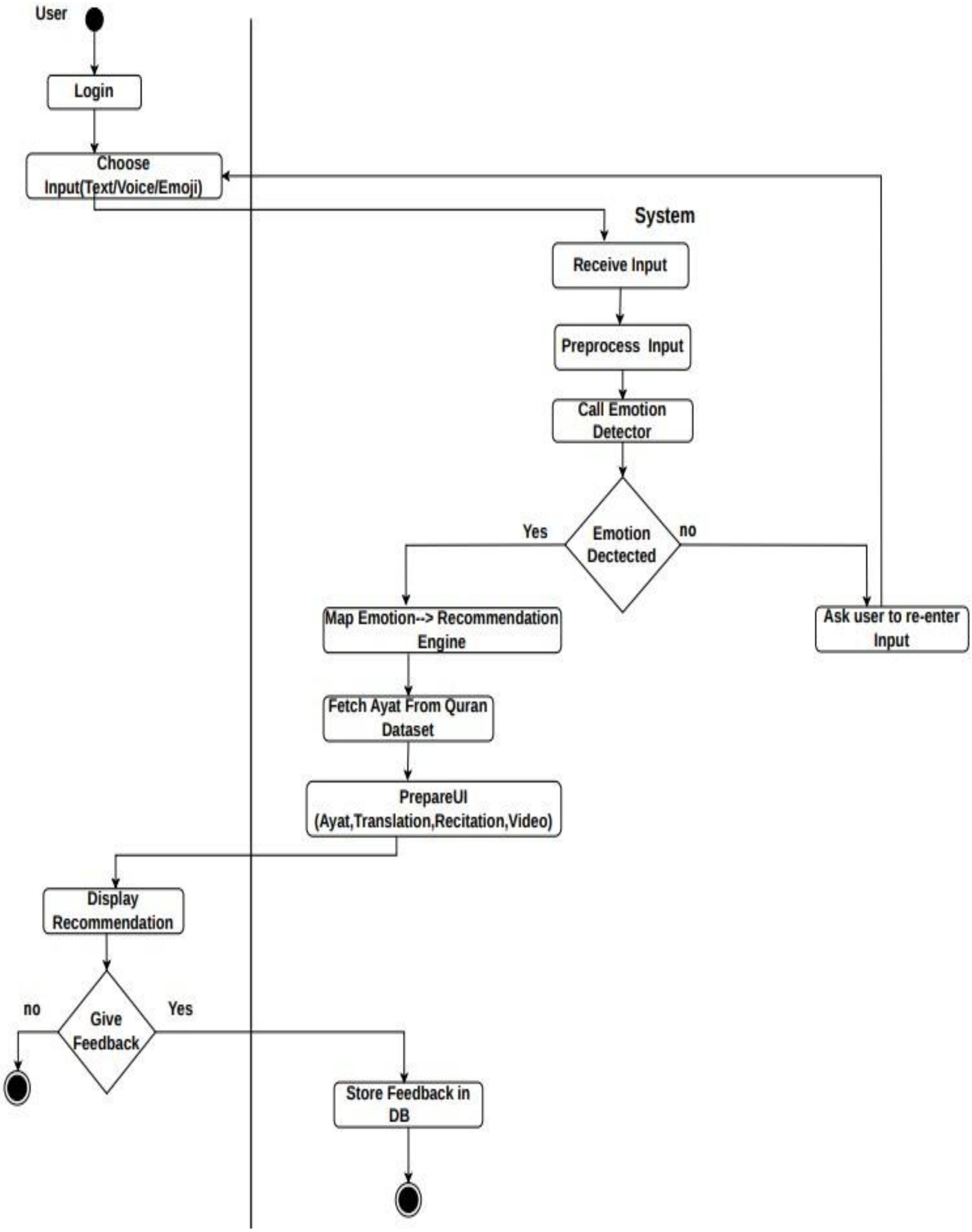

Figure 4. 8:Activity Diagram

4.4.2 Collaborative Diagram

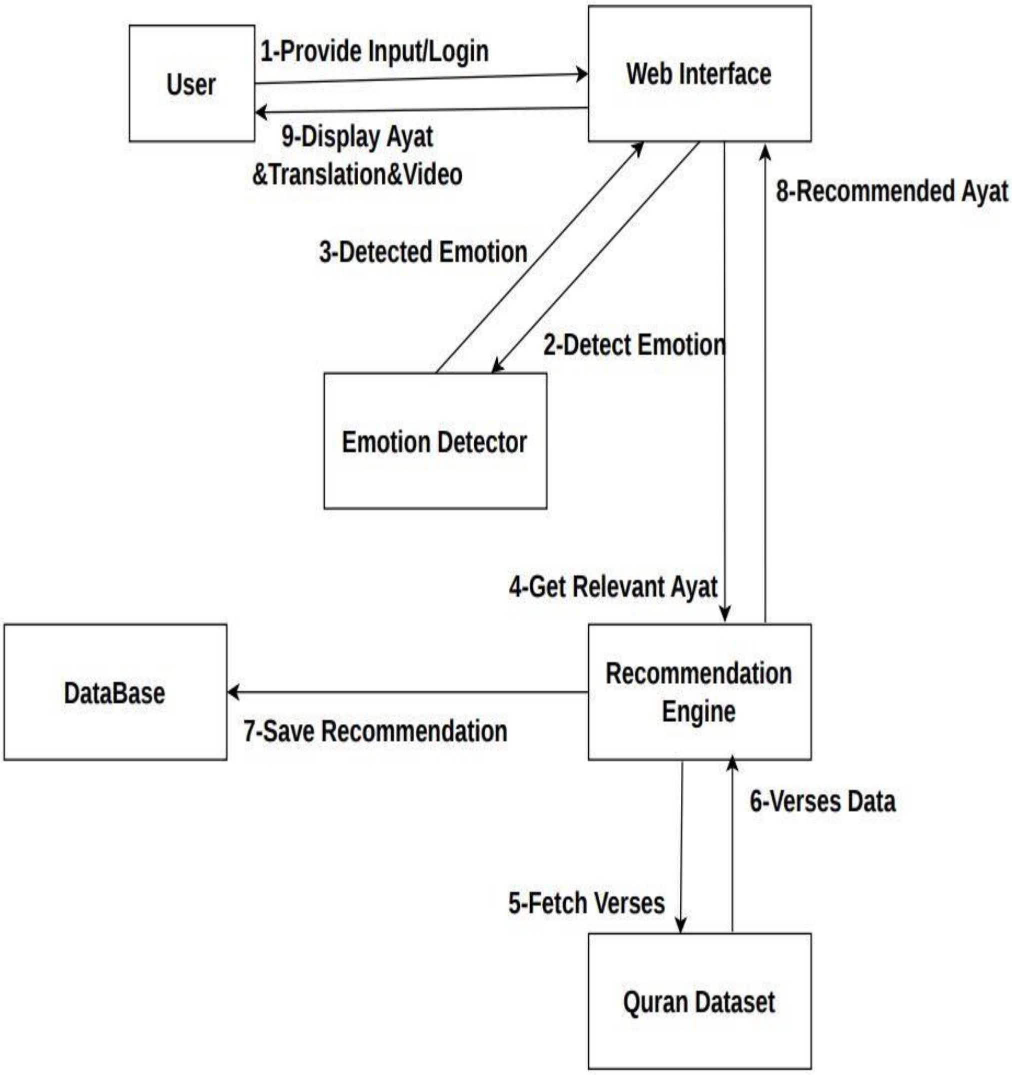

Figure 4. 9:Collaborative Diagram

4.5. UML Interaction Diagrams

4.5.1 Sequence Diagrams

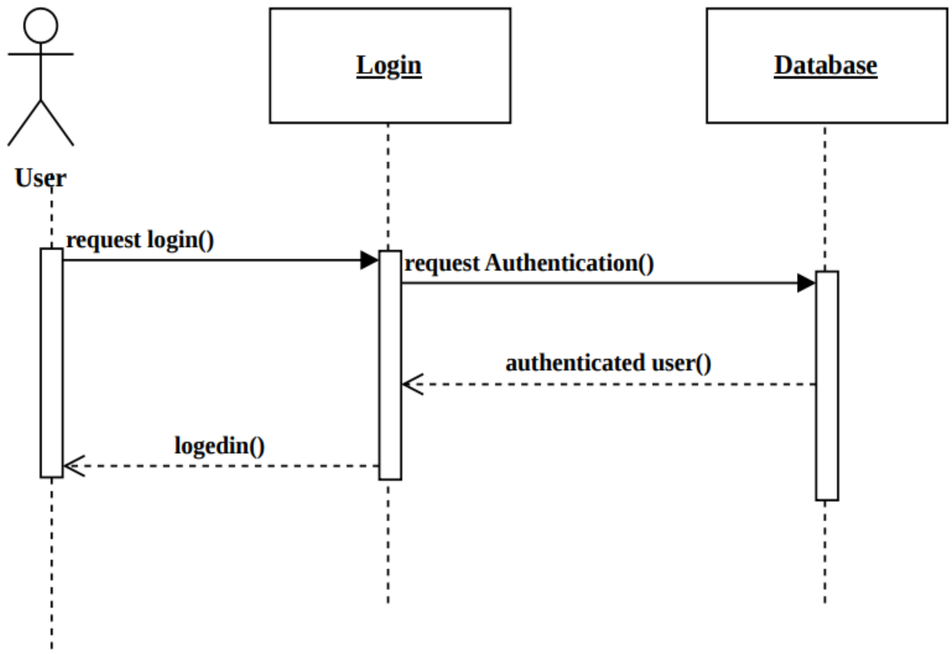

Figure 4. 10:User Login Sequence

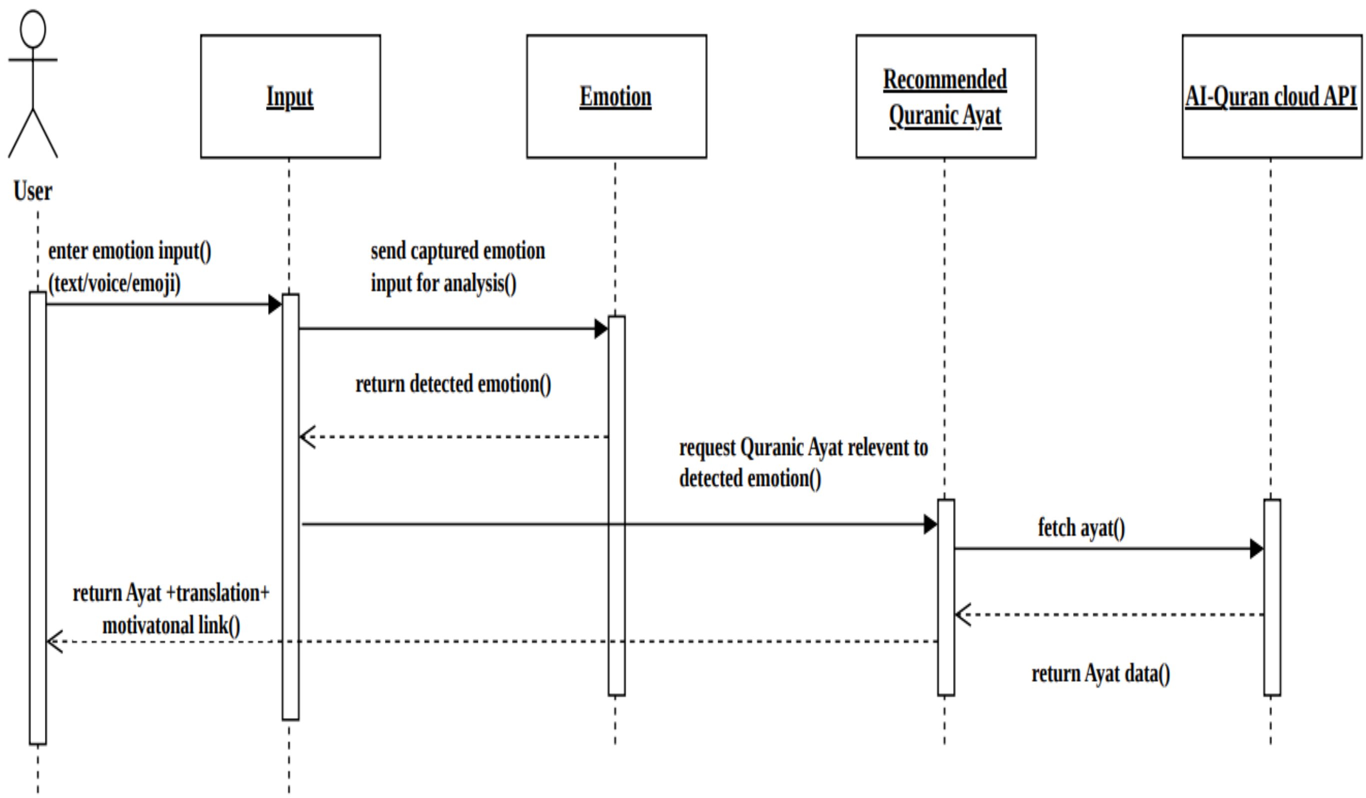

Figure 4. 11:User Input Sequence

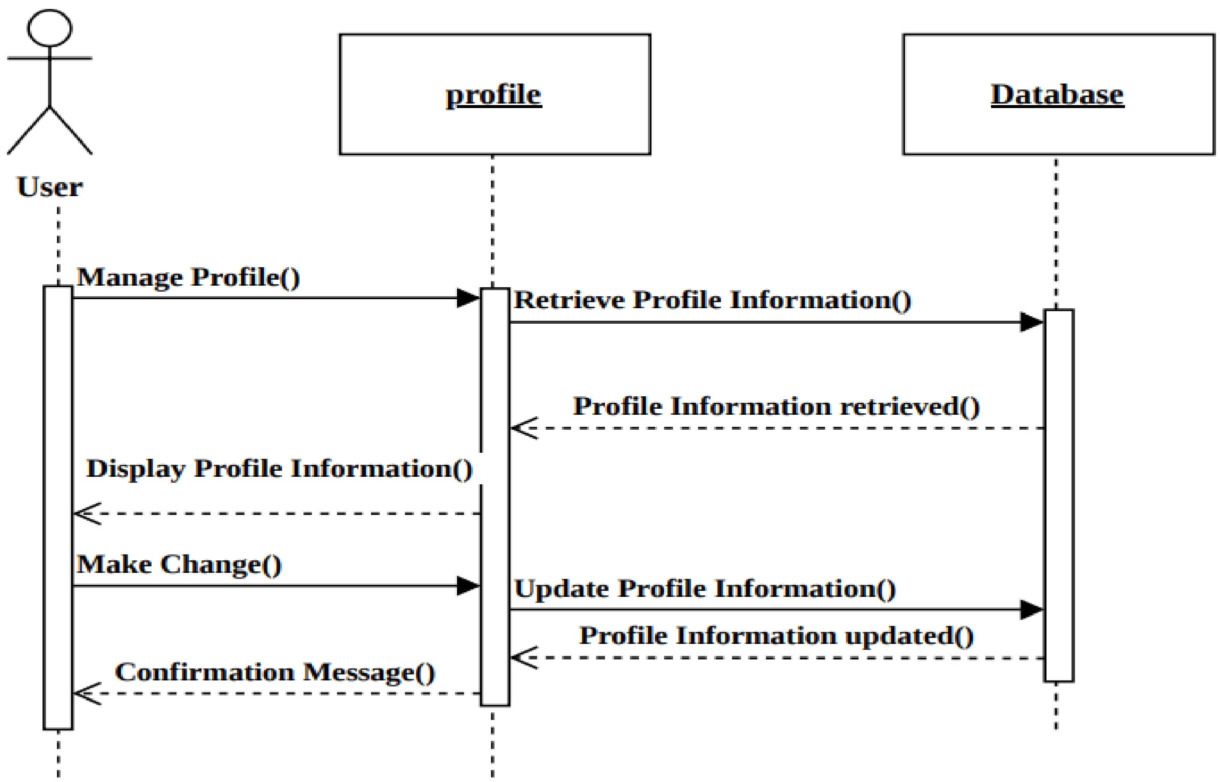

Figure 4. 12:User profile Sequence

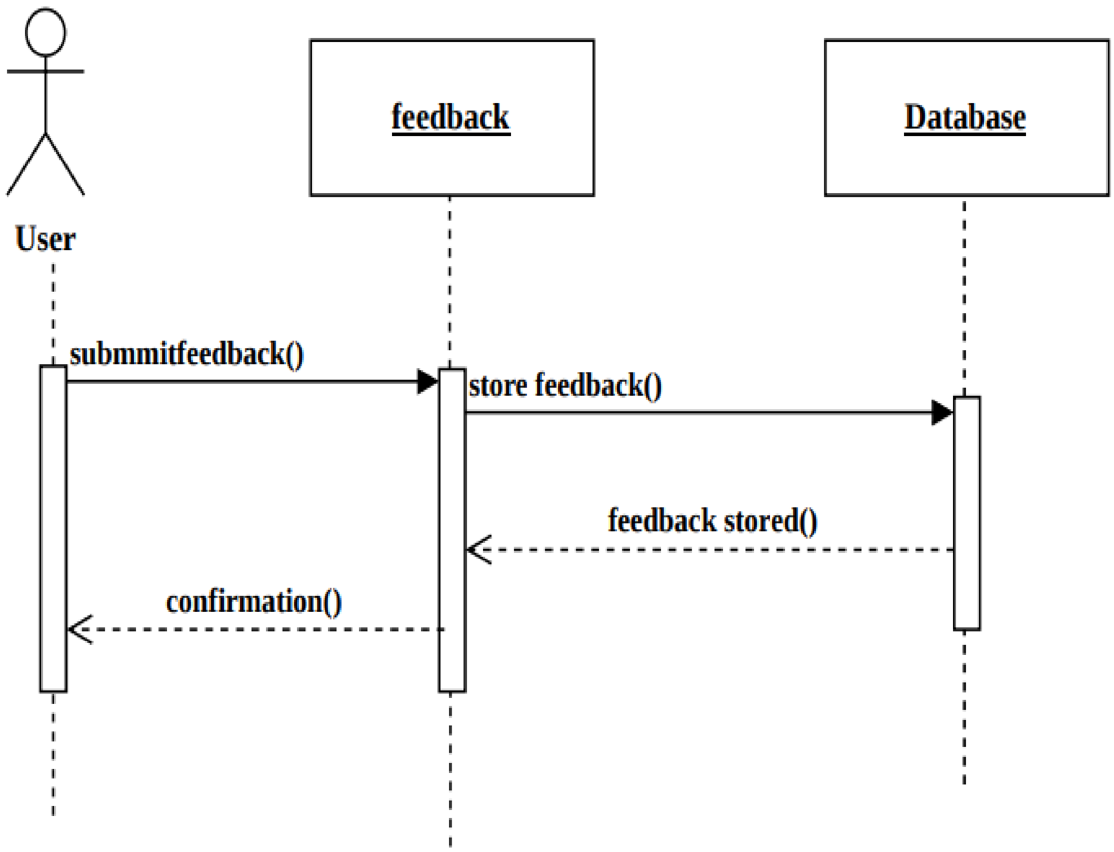

Figure 4. 13:User Feedback Sequence

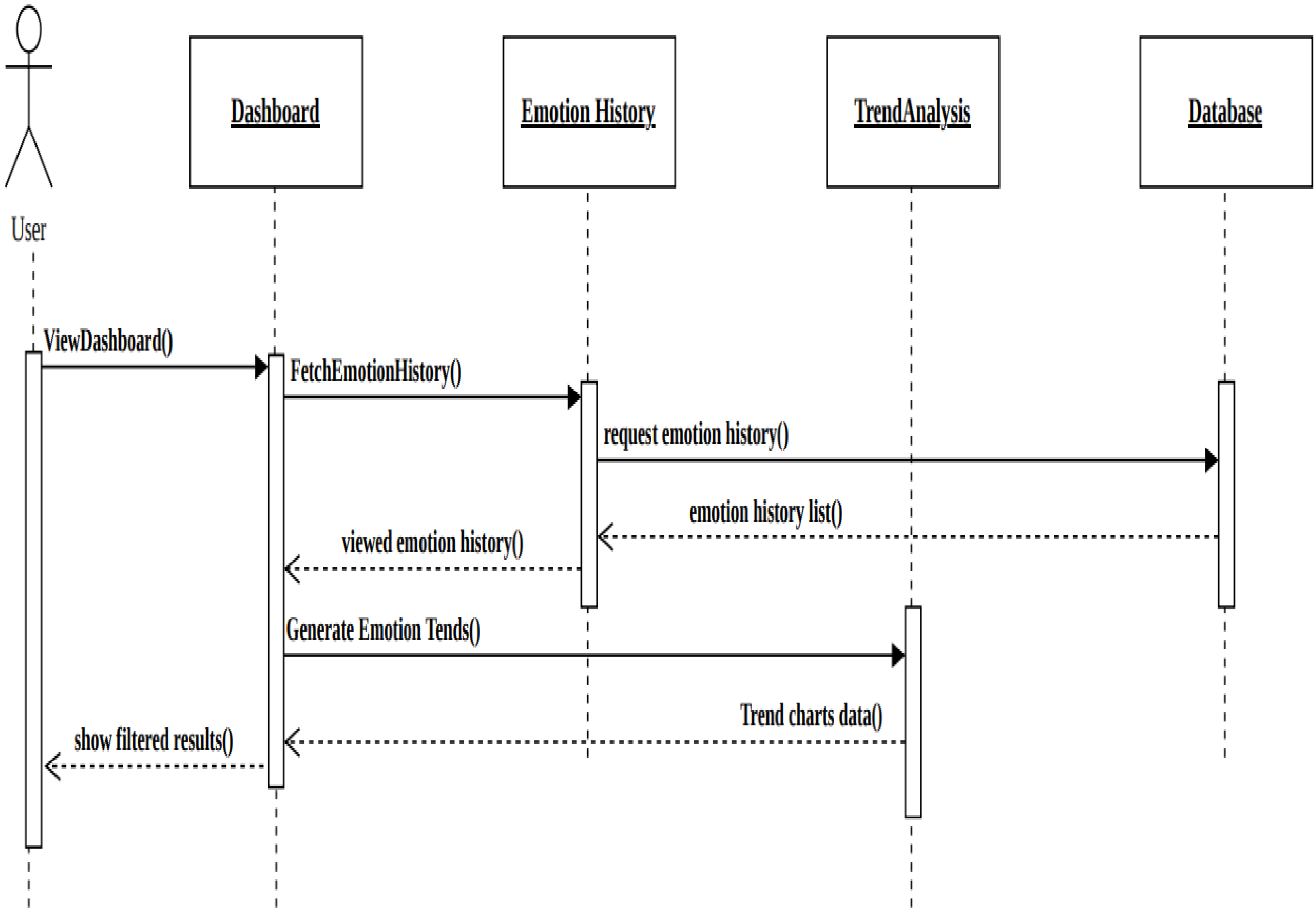

Figure 4. 14:User Dashboard Sequence

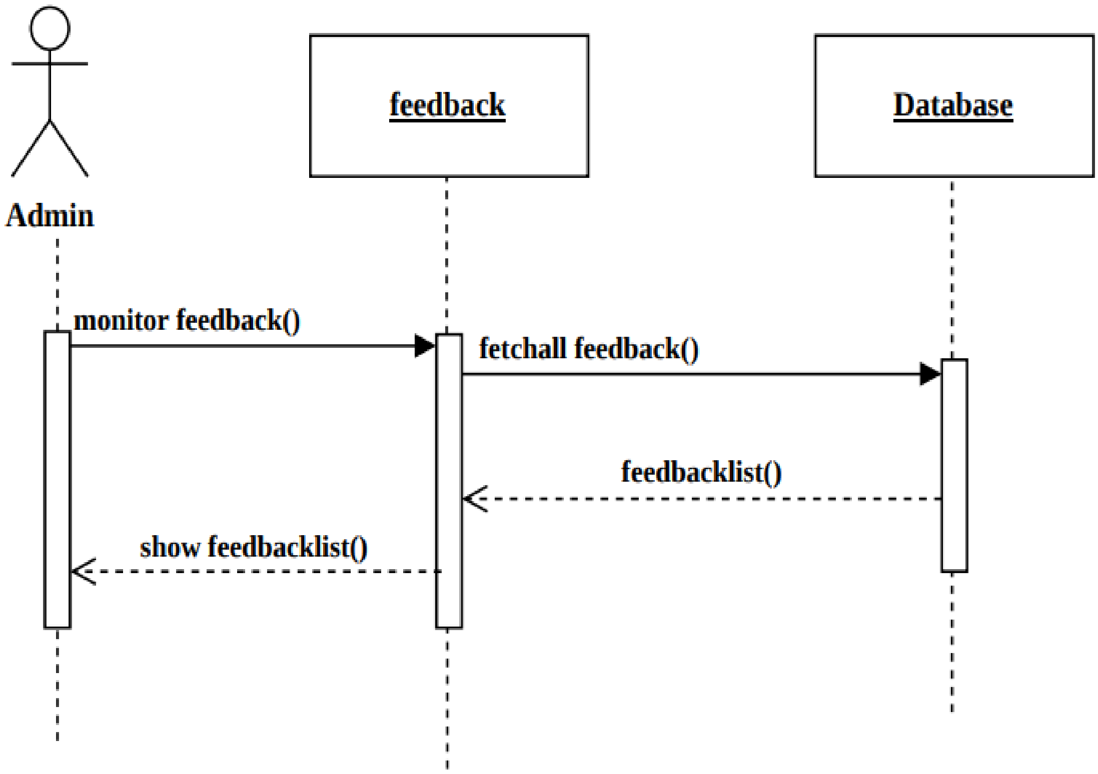

Figure 4. 15:Admin Feedback Sequence

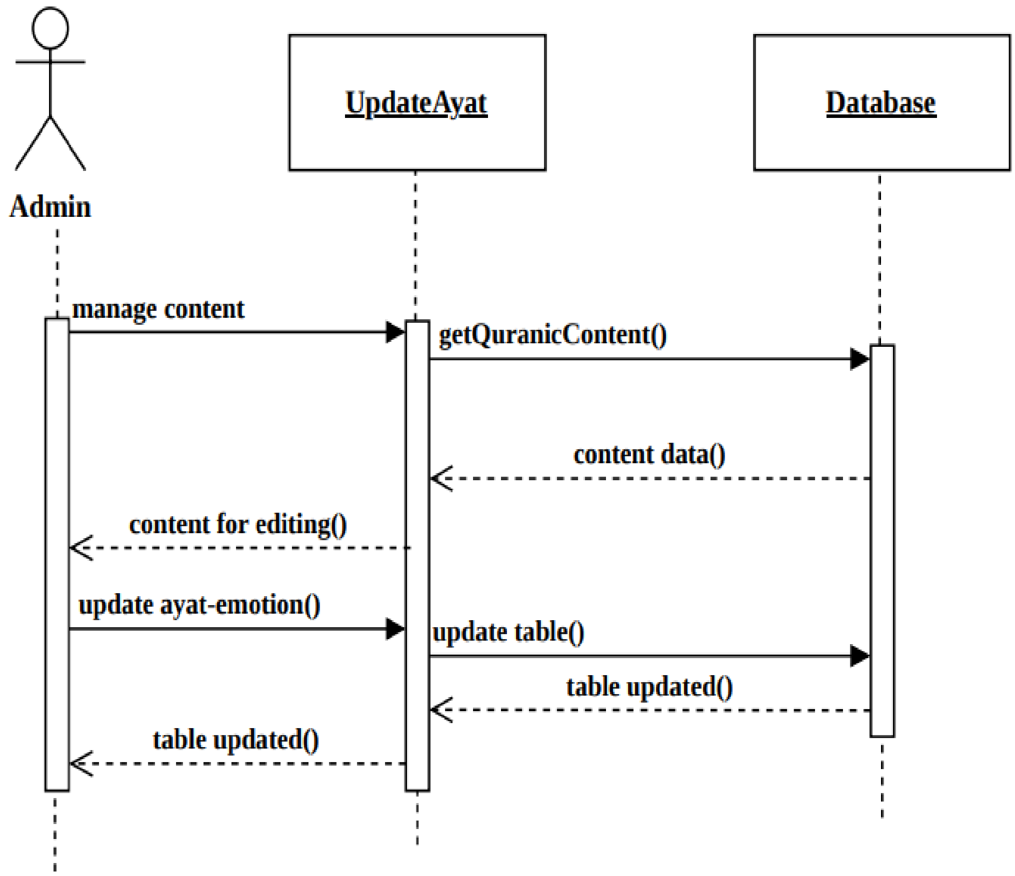

Figure 4. 16:Admin Update Sequence

Chapter 5: Implementation

This chapter will describe network protocols, user interface, component diagrams and choice of object middleware for the project. A network protocol is a standardized set of rules that define the communication process between devices within a network. These rules dictate how data is transmitted, received, and interpreted by the devices involved. By adhering to these protocols, devices can effectively communicate and exchange information, regardless of any variations in their internal processes, structures, or designs. A component diagram, sometimes known as a UML component diagram, is a visual representation that depicts the arrangement and organization of the physical components comprising a system. These diagrams are commonly employed to aid in the modeling of implementation details and to ensure that all essential functions of the system are addressed in the planned development.

5.1. Component Diagram

The component diagram of the Emotion-Based Quranic Recommendation System shows how different software components interact with each other. The system follows a modular architecture**,** where each feature (input, emotion detection, recommendation, Quran API, dashboard, admin panel, etc.) is organized into separate components. This improves maintainability, scalability, and makes future updates easier.

5.1.1 User Component Diagram

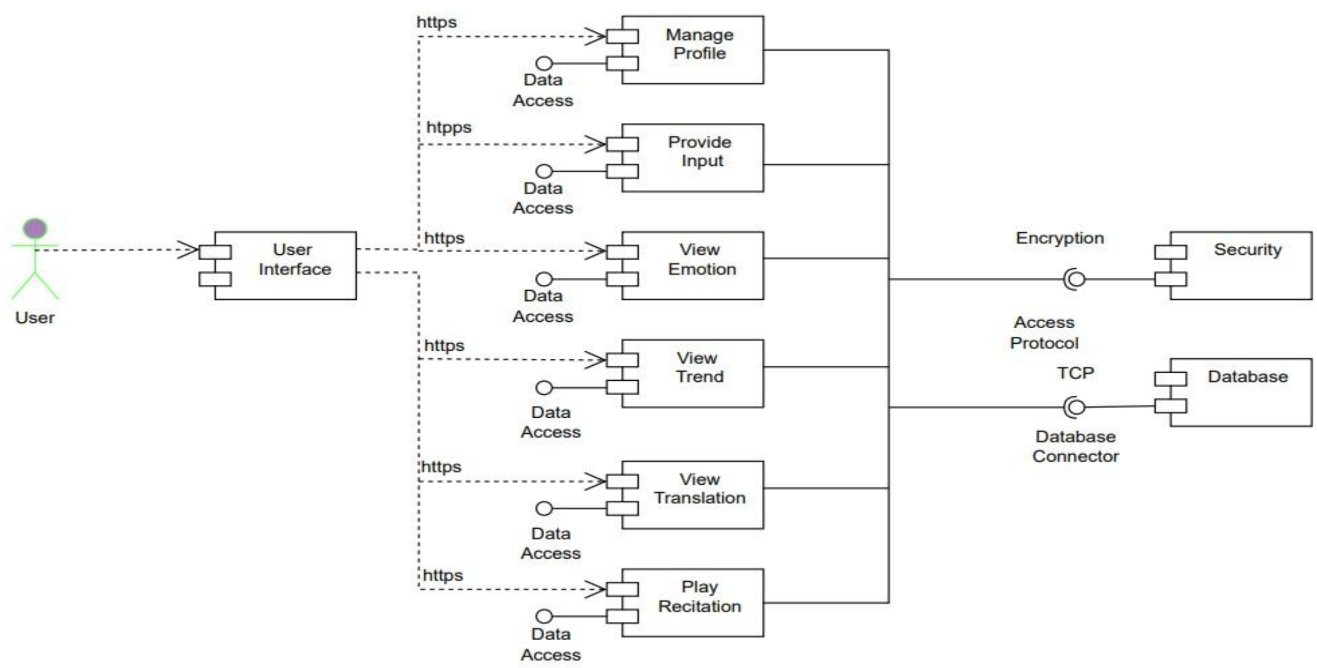

Figure 5. 1:User Component Diagram

- **User Interface**: Allows users to enter input and view results.
- **Provide Input:** Users can provide three types of inputs
- **Emotion Detection Components**

Text Model: Processes English input using DistilRoBERTa (HuggingFace). Urdu and Arabic inputs use a keyword fallback with negation and sarcasm handling.

Voice Model: Processes user speech using speech Recognition to classify emotion.

Emoji Mapping: Maps emojis to their corresponding emotional categories.

- **Emotion Detection Engine:** Combines results from text, voice, and emoji models and finalizes the user's emotion.
- **Recommendation System Components**

Recommendation Engine: Uses detected emotion to fetch relevant Quranic Ayat.

5.1.2. Admin Component Diagram

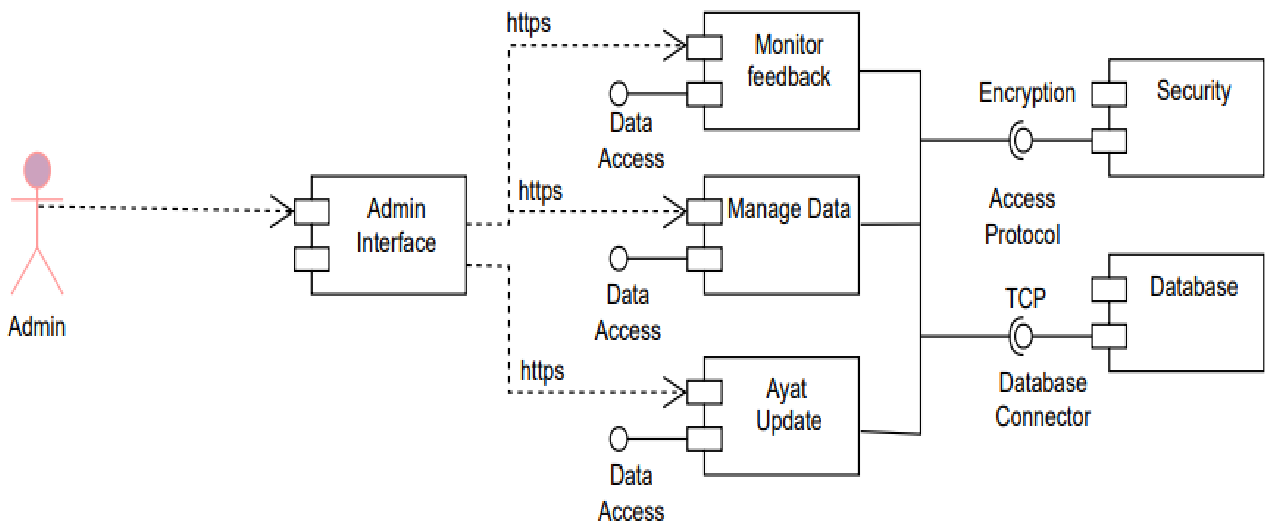

Figure 5. 2:Admin Component Diagram

- **Admin Components**

Admin Interface: Dashboard for managing system settings and data.

Manage Data: Admin can update: Translations, Recitation, Ayat Update.

- **Monitor Feedback**: Admin reviews user feedback and system performance.
- **User Management:** Admin manages user accounts and permissions.

- **Security Components**

Authentication Service: Handles user login, registration, and secure access.

- **Profile Service**: Ensures secure profile editing and data access.
- **Encryption**: Encrypts user input, stored data, and sensitive information.
- **Access Protocols (HTTPS):** Ensures secure communication between all connected components.

5.2. Network and Protocol Choice

In our Emotion-Based Quranic Recommendation System, network protocols play an important role in ensuring smooth and secure communication between the user interface, backend server, Quran API, and the database. A network protocol is simply a set of rules that controls how data is sent, received, and processed across the internet. Without these rules, different system components would not understand each other, and communication would fail.

Our project is a web-based system that relies on AI models, real-time communication, and external Quran APIs, choosing correct protocols is necessary for fast, secure, and reliable operation.

5.2.1. HTTPS

HTTPS is used for all communication between the users browser and the server.
It ensures that:

- Data is encrypted
- User input (text, voice, emoji) is secure
- Recommended Ayat and translations are safely delivered

This is important because users share personal emotions, and their data must remain private.

5.2.2. REST API Protocol

Our system connects with external Quran APIs like Al-Quran Cloud using REST API calls.
This allows us to retrieve:

- Quranic Ayat
- Translations
- Recitations
- Tafseer

REST protocol ensures structured and fast communication between our backend and external APIs.

5.3. Choice of Object Middleware

Object middleware is the software layer that helps different components of a system communicate with each other smoothly. It acts as a bridge between the front-end (React website), back-end server (Python/Flask or Node.js), AI emotion models, and the database. It ensures data is sent and received in a structured, secure, and efficient way.

For our project, we selected middleware tools that support fast communication**,** scalability, and integration with machine learning models**.**

5.3.1. RESTful API Middleware (Backend Framework)

We use REST API communication between:

- Frontend (React)
- Backend (Flask / FastAPI / Node.js)
- AI Emotion Models (BERT, speech Recognition)
- Quran API Services

REST middleware allows the system to send and receive data in a standard JSON format.
It ensures:

- Fast communication
- Easy debugging
- Secure interaction
- Separation of frontend and backend logic

5.4. User Interface

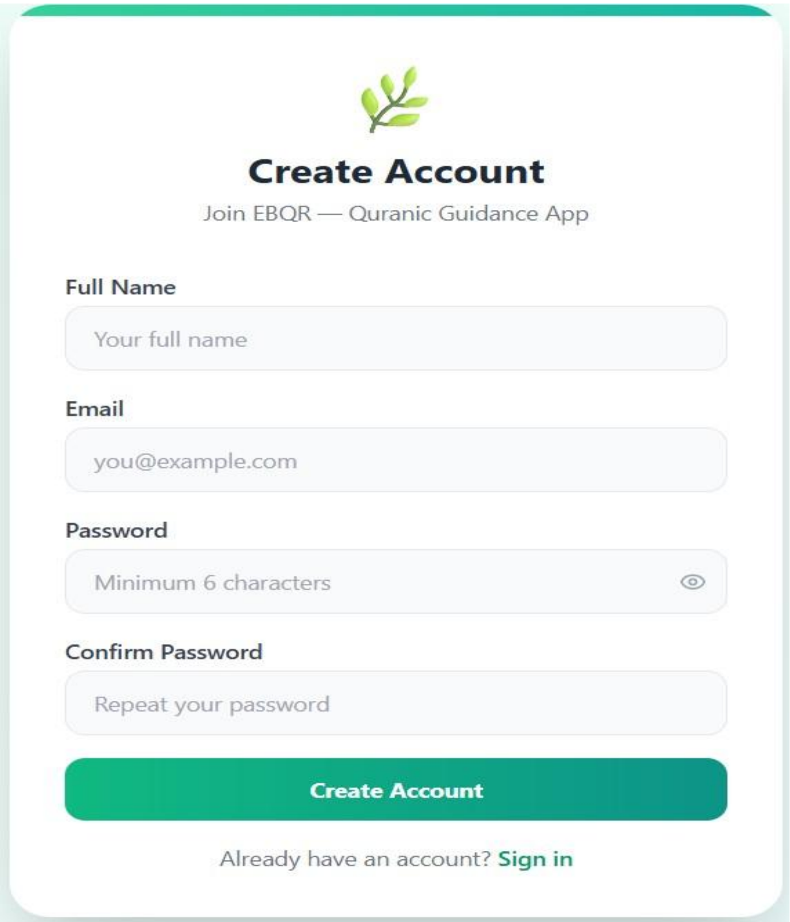

Figure 5. 3:User Registration

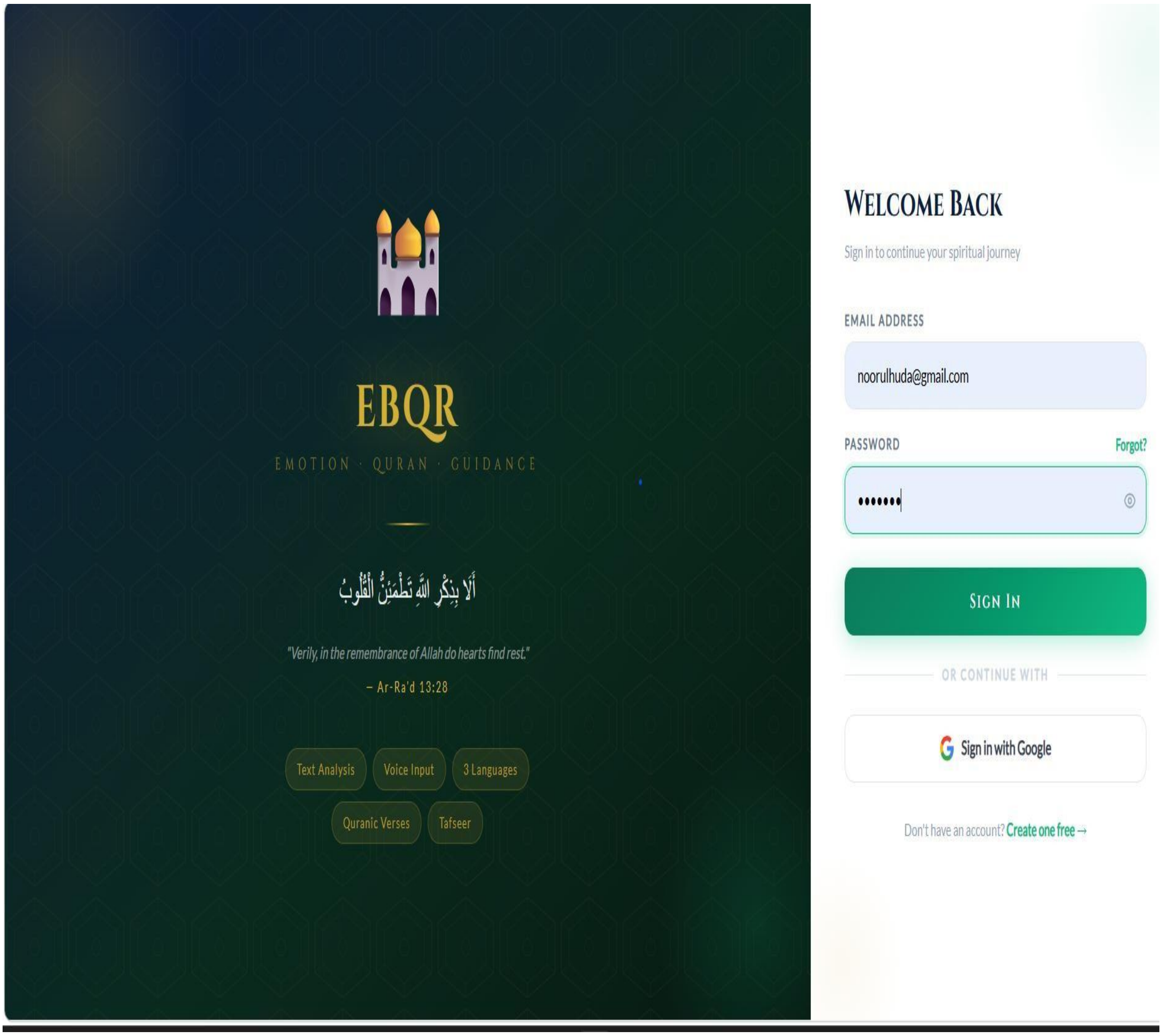

Figure 5. 4:User Login Page

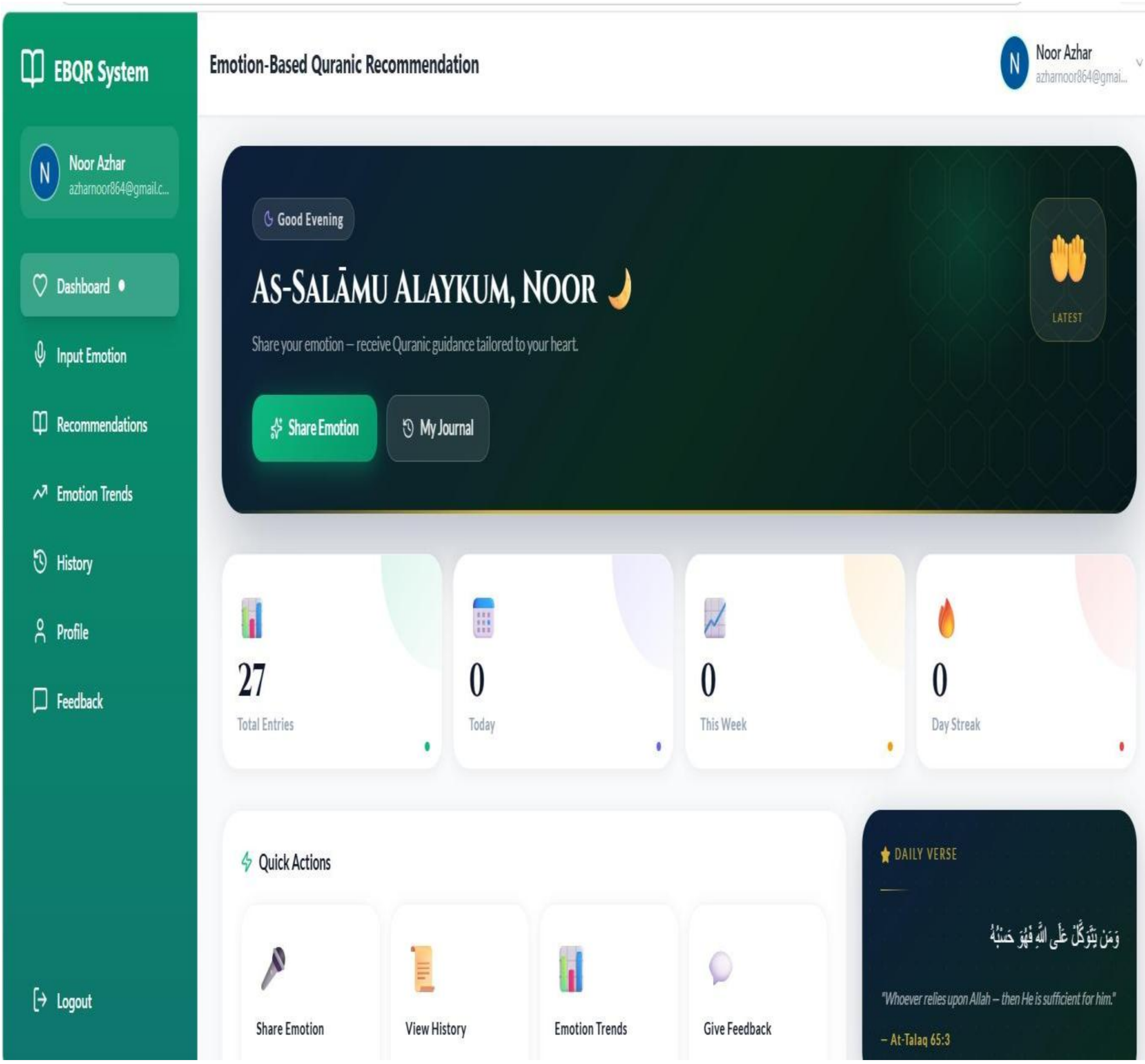

Figure 5. 5:Dashboard

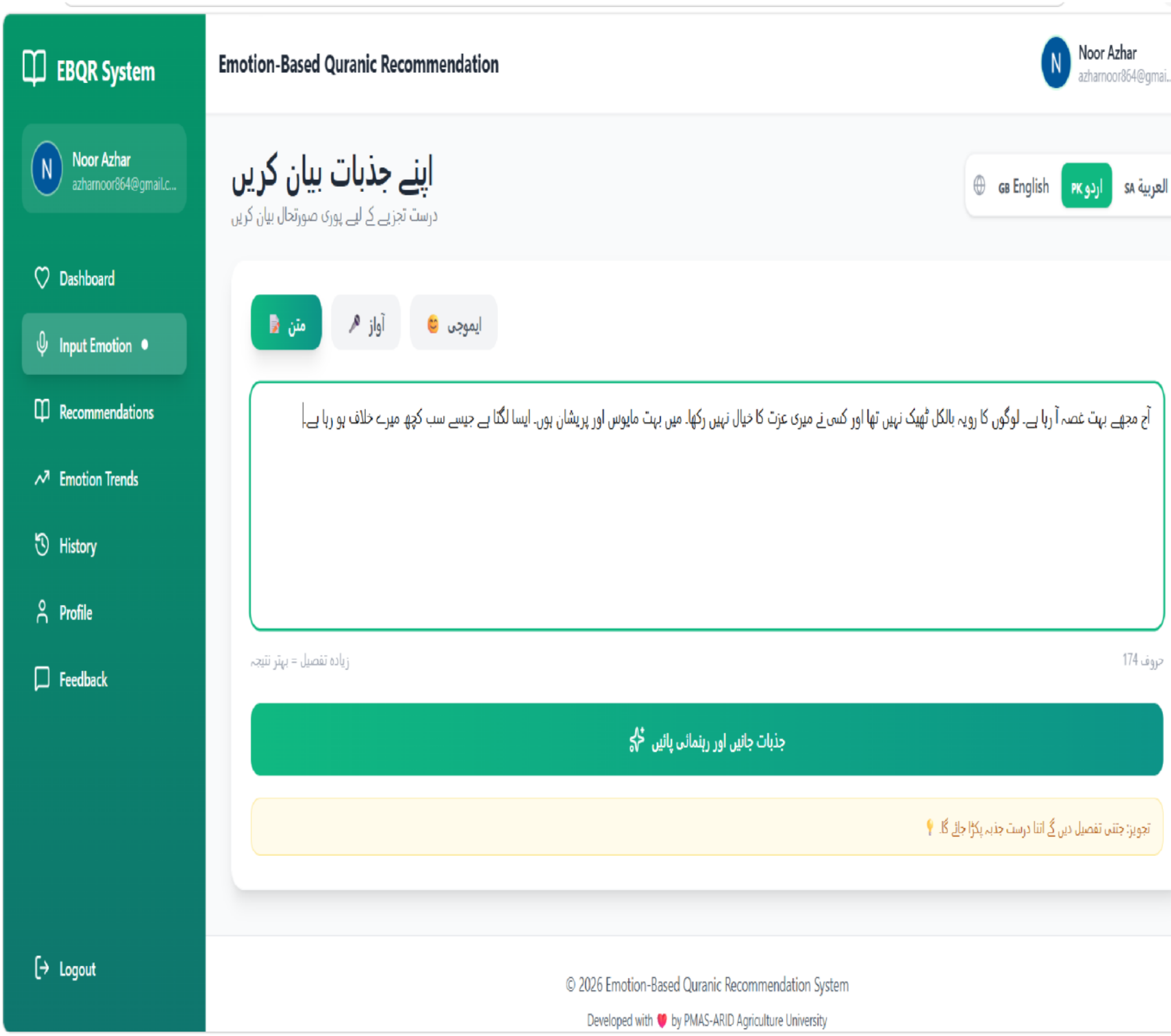

Figure 5. 6:User Input

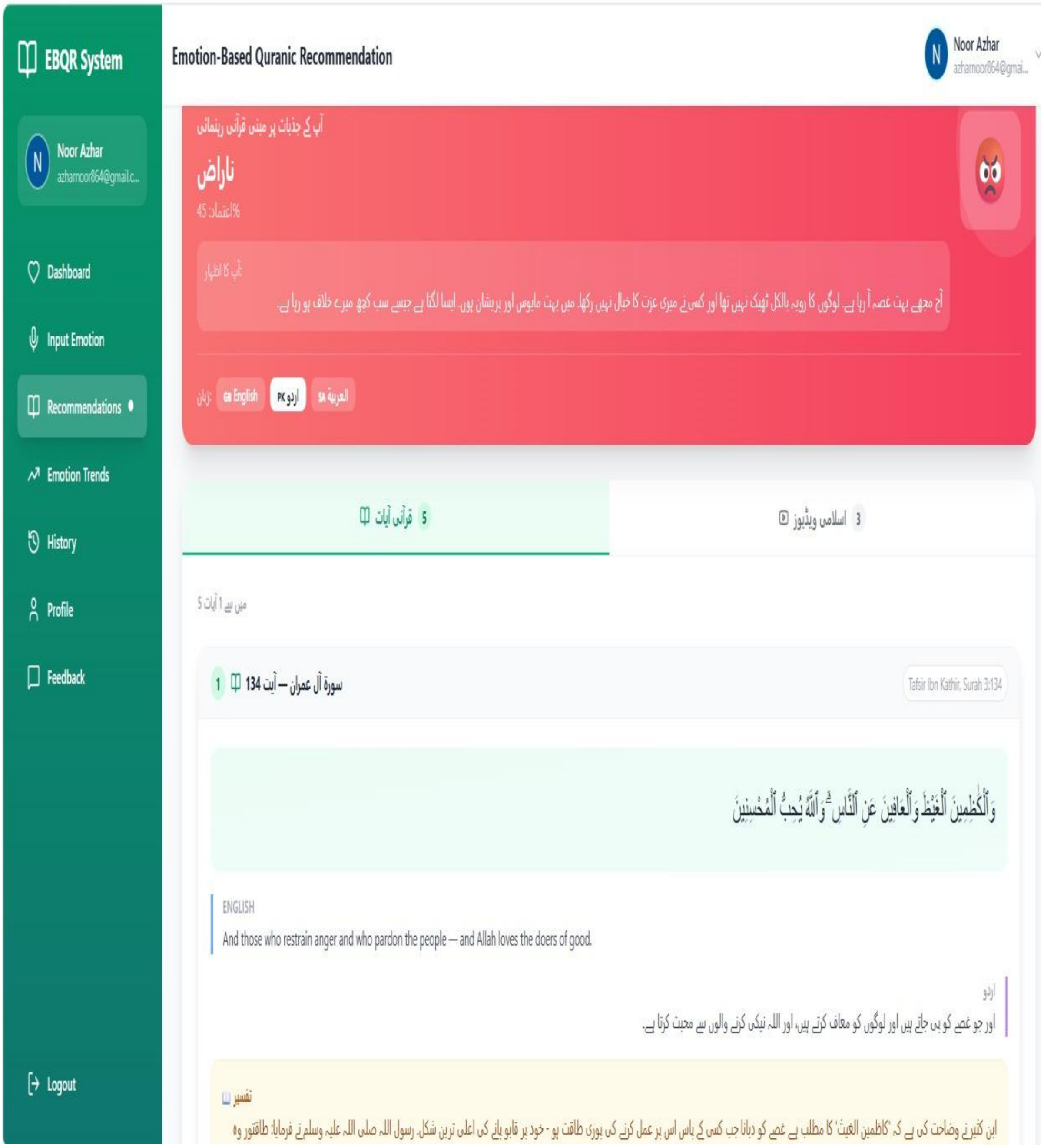

Figure 5. 7:Emotion Detection and Ayat Recommendation

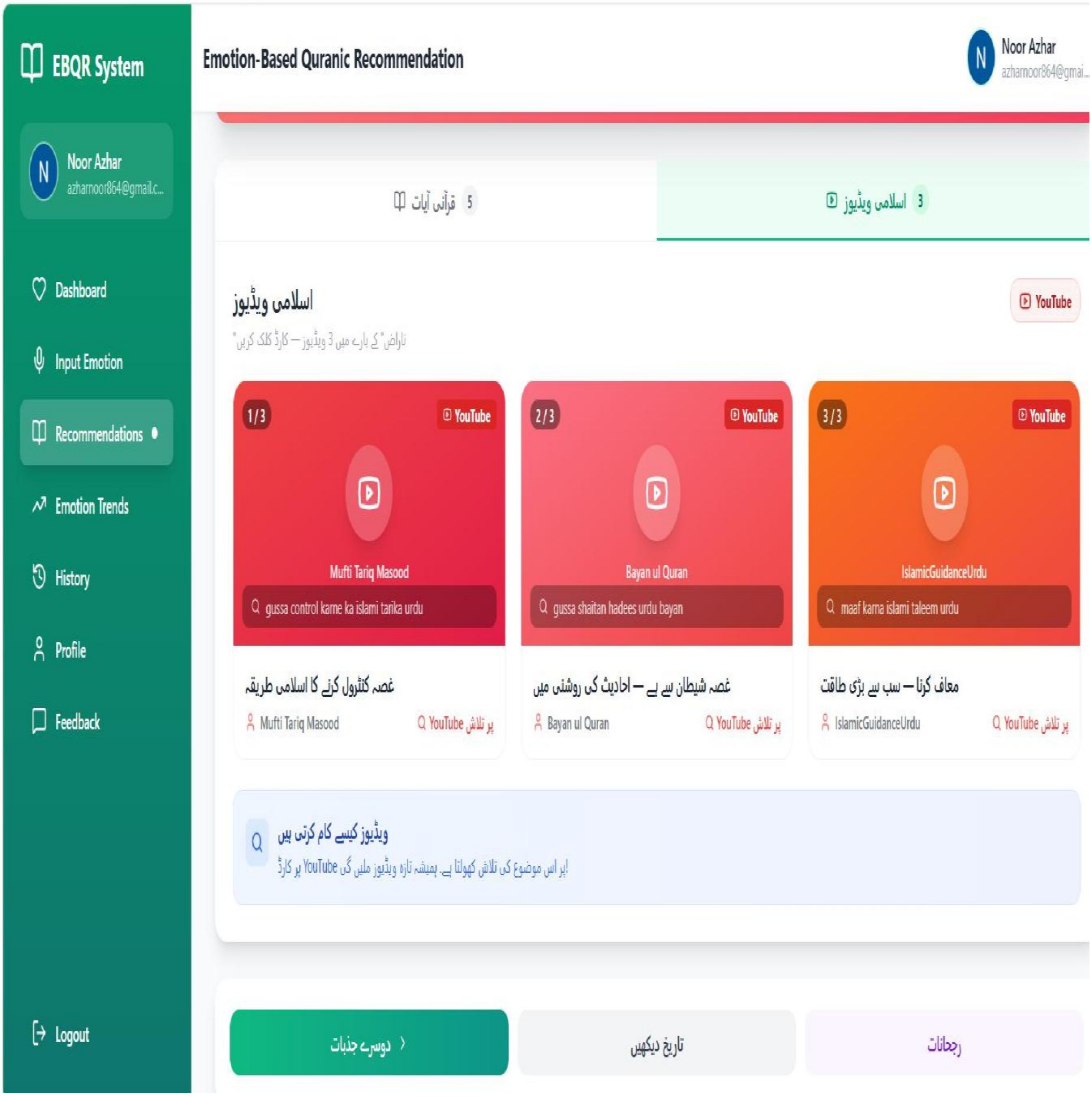

Figure 5. 8:Motivational Video

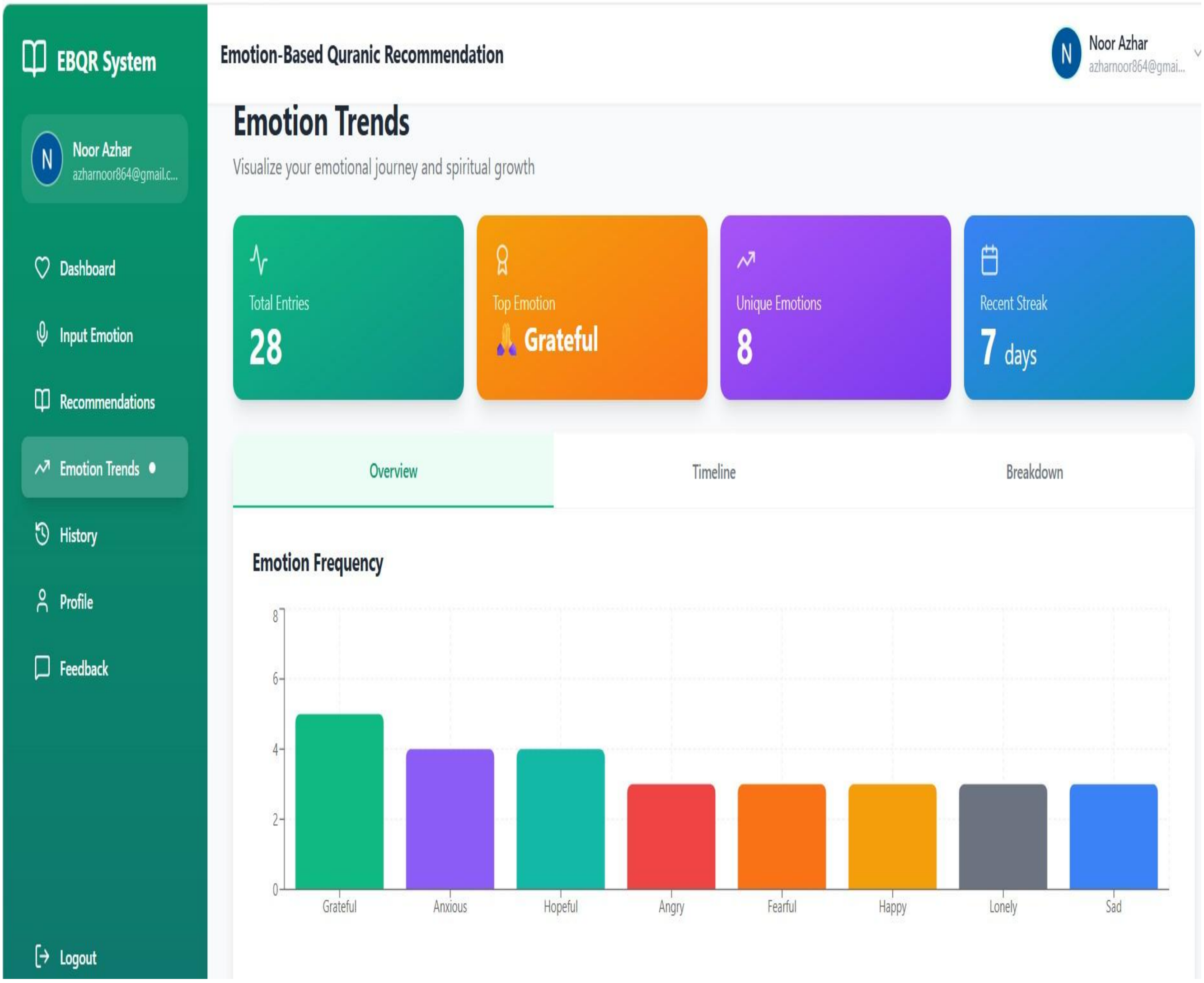

Figure 5. 9:Emotion Trend

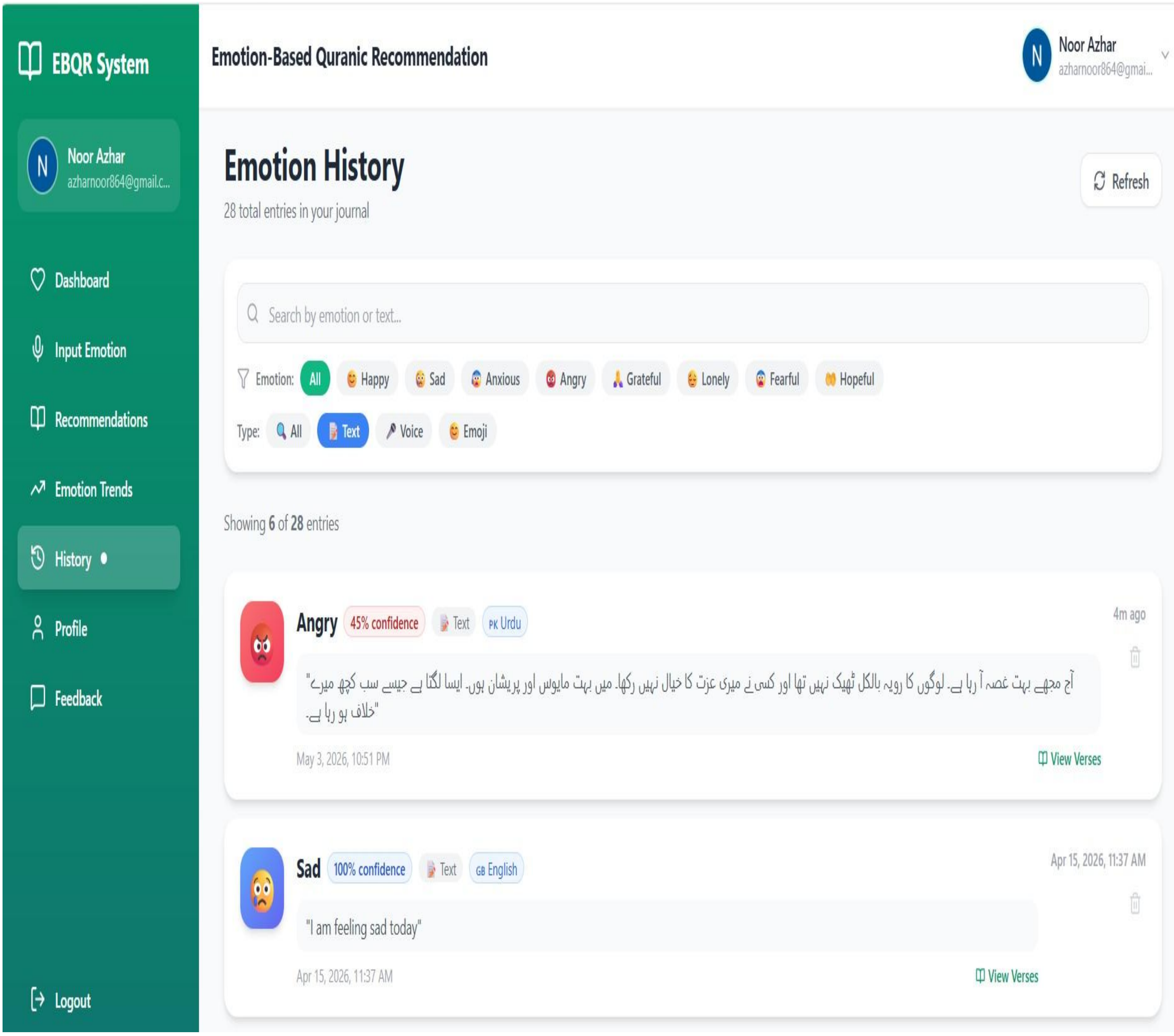

Figure 5. 10:Emotion History

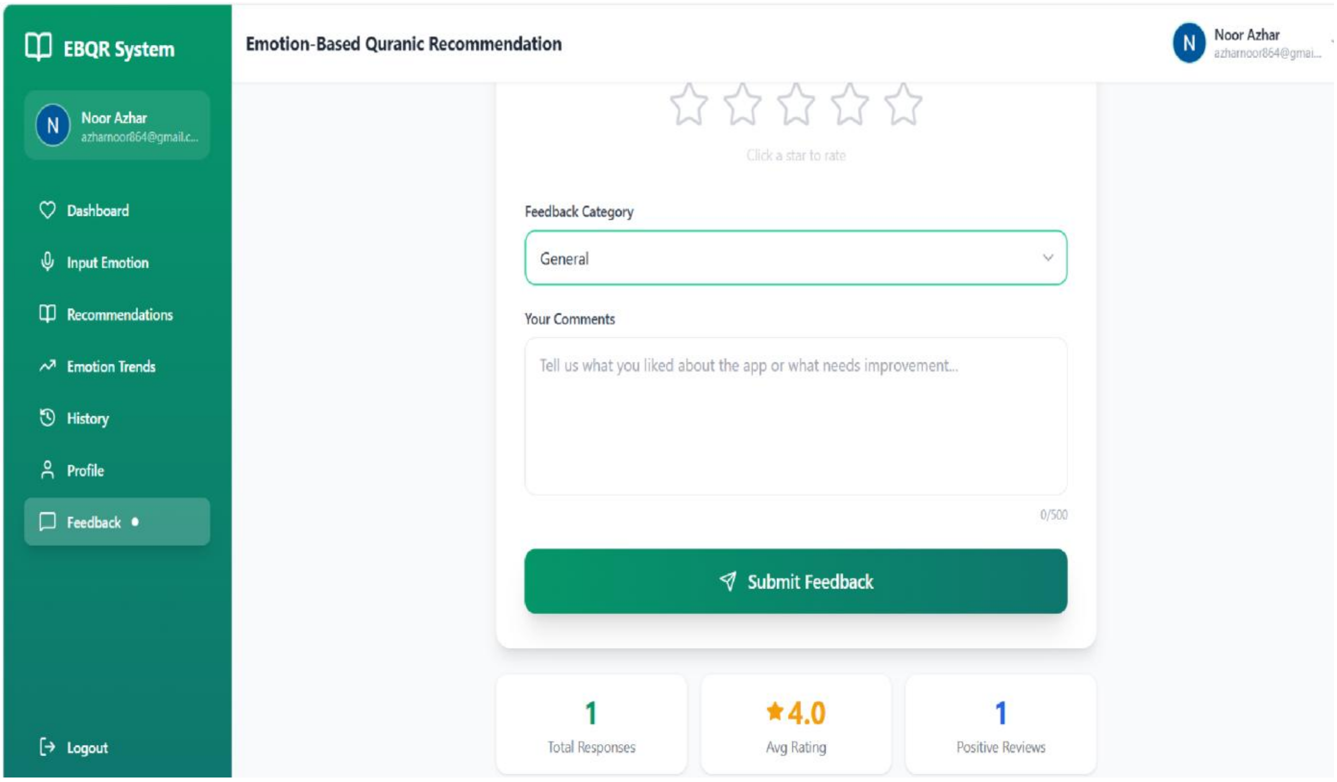

Figure 5. 11:Feedback

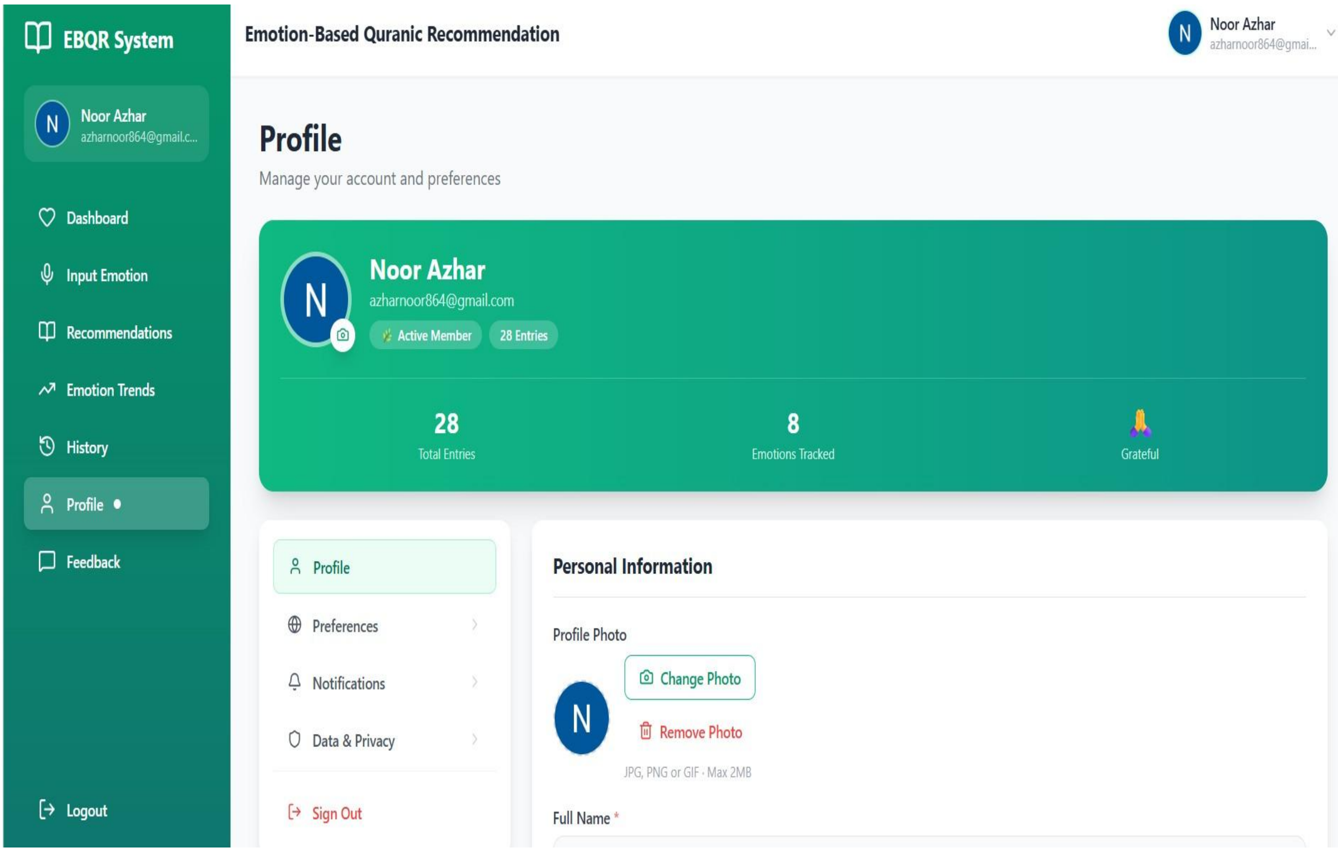

Figure 5. 12:User Profile

Chapter 6: Testing and Evaluation

This chapter presents a comprehensive set of testing strategies, evaluation results, and performance analysis conducted for the Emotion-Based Quranic Recommendation System (EBQR). Testing phases were performed to verify that the system correctly detects user emotions across three input types (text, voice, and emoji), retrieves relevant Quranic verses, manages user accounts securely, and delivers a smooth user experience. The chapter covers verification, validation, usability testing, unit and integration testing, system and acceptance testing, as well as stress testing and hardware configuration. It concludes with a formal evaluation of the system's performance, followed by deployment and maintenance activities.

6.1. Verification

Verification in EBQR ensures that the system is built correctly according to its specified requirements and design documents. It involves reviewing code, architecture, and test outcomes to confirm that each module confirms to defined standards and expected behaviour before integration.

Verification activities covered the following areas throughout the development lifecycle:

- Requirement traceability review confirming all functional and non-functional requirements in Chapter 3 are implemented in the Flask backend and React frontend.
- Design conformance review verifying that actual module structure matches the three-tier architecture (Presentation, Application, Database) described in Chapter 4.
- Code inspection of app.py, emotion_detector.py, database_helper.py, quran_service.py, and all React page components.
- API route review ensuring every frontend api.js call has a corresponding validated Flask endpoint with proper JWT authentication.
- Database schema review confirming all tables (users, emotion_history, ayat_recommendations, user_feedback) correctly store and retrieve required data fields.

These activities ensured that errors were detected early during development sprints, reducing costly fixes at later stages.

6.1.1 Functionality Testing

All critical functionalities of EBQR were subjected to functional testing. Developers validated each feature defined in the requirement specification to confirm correct and complete implementation.

The following core functionalities were tested:

- **User Registration and Login:**

  The system correctly registers new users, hashes passwords using werkzeug.security (PBKDF2-SHA256), generates JWT tokens, and stores user data in MySQL without errors.

- **Text Emotion Detection:**

  The DistilRoBERTa-based transformer model correctly classifies English text inputs into one of eight emotions. Urdu and Arabic inputs fall back to keyword matching correctly.

- **Voice Emotion Detection:**

  Audio files (WAV, MP3, OGG, WebM) are transcribed using the Speech Recognition library with the Google Web Speech API, and the resulting text is classified by the DistilRoBERTa model to produce an emotion label and confidence score.

- **Emoji Emotion Detection:**

  The emoji mapping dictionary correctly maps 30+ emoji characters to their corresponding emotion categories with appropriate confidence scores.

- **Quranic Verse Recommendation:**

  After emotion detection, the Quran Service correctly retrieves 5 relevant verses per emotion in the requested language (English, Urdu, or Arabic).

- **Emotion History Storage:**

  Detected emotions and their associated recommendations are correctly saved to emotion history and ayat_recommendations tables with timestamps.

- **Profile Management:**

  Users can correctly update their name, language preference, avatar, and password. Security question setup and retrieval work correctly.

- **Forgot Password Flow:**

  All three steps (email lookup, security answer verification, password reset) operate correctly end-to-end.

- **Feedback Submission:**

  User feedback (rating, category, comments) is saved both locally in local Storage and to the MySQL database via the /api/feedback endpoint.

- **History Deletion:**

  Individual emotion history entries are correctly deleted from the database when the user confirms deletion on the History page.

- **Dashboard Statistics**:

  Total entries, emotion distribution, and streak calculations correctly reflect the user's actual history data.

6.1.2 Static Testing

Static testing was conducted without executing the system. All Python scripts and React components were reviewed for code quality, logic correctness, and adherence to project standards.

Code review activities included:

- Peer review of emotion_detector.py verifying the transformer model loading, emotion mapping logic, confidence score calculation, and multi-language fallback handling.
- Review of database_helper.py confirming execute_query() commit placement, connection handling, and correct return types for all database operations.
- Review of app.py verifying JWT token_required decorator, correct HTTP status codes, and proper error responses for all API endpoints.
- Review of api.js confirming correct API route paths, proper Authorization header attachment, and consistent success/error response normalization.
- Review of all React page components for correct state management, API call patterns, and loading/error state handling.

Documentation review confirmed that all design diagrams in Chapter 4 accurately reflect the implemented system architecture. Standards compliance was verified against PEP 8 for Python files and React best practices for frontend components.

6.2. Validation

Validation confirms that the final EBQR system fulfils the actual user needs and project objectives. It answers the question: 'Are we building the right product?' Validation was performed by executing structured test cases and comparing actual results against expected outcomes. Five formal test cases were designed and executed, as presented below.

Table 6. 1:Test Case: User Registration and Authentication

Field	Details
Test Case ID	TC-001
Test Case Name	User Registration and Authentication

Description	Verify that new users can register with valid credentials, receive a JWT token, and successfully log in.
Prerequisite	MySQL database running. users table created. Flask server active on port 5000.
Steps	Send POST /api/auth/register with name, email, password. Verify 201 response with token. Send POST /api/auth/login with same credentials. Confirm JWT token returned and stored in local Storage.
Expected Result	Registration succeeds with HTTP 201. Login returns valid JWT token. User object includes user_id, name, email, language.
Actual Result	Registration and login both succeeded. Token stored correctly. Navbar updated with user name and initials.
Pass / Fail	Pass

Table 6. 2:Test Case: Text Emotion Detection

Field	**Details**
Test Case ID	TC-002
Test Case Name	Text Emotion Detection and Quranic Recommendation
Description	Validate that text input is correctly classified into an emotion and mapped to relevant Quranic verses.
Prerequisite	Dual emotion detection models loaded (j-hartmann and XLM-RoBERTa). langdetect installed. quran_emotion_dataset.csv present. User authenticated.
Steps	Submit text input "I feel very sad and alone today" via /api/detect. Observe emotion label returned. Confirm confidence score between 0 and 1. Verify 5 Quranic verses returned in selected language.
Expected Result	Emotion "Sad" detected with confidence >= 0.70. Five relevant Quranic verses returned with Arabic text, English translation, and tafseer.
Actual Result	Emotion "Sad" returned with confidence 0.87. Five verses retrieved successfully with correct Arabic, English, and Urdu fields populated.
Pass / Fail	Pass

Table 6. 3:Test Case: Voice Emotion Detection

Field	**Details**
Test Case ID	TC-003
Test Case Name	Voice Emotion Detection via Audio Upload
Description	Verify that audio recordings are transcribed by Speech Recognition (Google Web Speech API) and the resulting text is classified into an emotion by the j-hartmann DistilRoBERTa model (English) or XLMRoBERTa with keyword scorer (Urdu/Arabic).
Prerequisite	Voice emotion model loaded. Audio upload folder configured. Supported formats: WAV, MP3, OGG, WebM.
Steps	Record a 3-5 second voice clip expressing sadness. Submit audio to /api/detect/voice. Verify emotion label returned. Confirm file deleted from server after processing.
Expected Result	Audio processed without errors. Valid emotion label returned. Confidence score provided. Uploaded file removed from server after processing.
Actual Result	Voice input correctly processed. Emotion "Sad" returned. Temporary file deleted. History entry created in database.
Pass / Fail	Pass

Table 6. 4:Test Case: Emoji Emotion Detection

Field	**Details**
Test Case ID	TC-004
Test Case Name	Emoji Emotion Detection and Mapping
Description	Validate that emoji inputs are correctly mapped to emotion categories and produce relevant Quranic recommendations.
Prerequisite	Emoji mapping dictionary loaded in emotion_detector.py. User authenticated.
Steps	Submit emoji input with crying face emoji via /api/detect. Verify emotion returned as "Sad". Submit praying hands emoji. Confirm "Grateful" emotion returned.

Expected Result	Crying face emoji maps to "Sad". Praying hands map to "Grateful". Confidence score of 0.90 returned for recognized emoji. Unrecognized emoji defaults to "Hopeful" with 0.50 confidence.
Actual Result	All tested emoji mapped correctly. Recognized emoji's returned 0.90 confidence. Unrecognized emoji defaulted to Hopeful correctly.
Pass / Fail	Pass

Table 6. 5:Test Case: Forgot Password Flow

Field	**Details**
Test Case ID	TC-005
Test Case Name	Forgot Password Three-Step Flow
Description	Verify that users can recover account access through the security question verification flow.
Prerequisite	User account exists with a security question and answer set. Flask server active.
Steps	Submit registered email to /api/auth/security-question. Verify security question returned. Submit correct answer to /api/auth/verify-answer. Confirm reset token returned. Submit new password to /api/auth/resetpassword.
Expected Result	Security question retrieved correctly. Correct answer produces a reset token. New password saves successfully. Old password no longer works. New password authenticates correctly.
Actual Result	All three steps completed successfully. Password updated in database. Login with new password confirmed working.
Pass / Fail	Pass

All five test cases passed successfully, confirming that the system's core functions operate as specified.

6.3. Usability Testing

Usability testing evaluated how intuitive and accessible the EBQR system is for its intended users: individuals seeking emotional and spiritual support through Quranic guidance. The primary interfaces under test were the Login, Register, EmotionInputPage, Recommendations Page, Dashboard, History Page, Trends Page, and Feedback Page React components.

Testing focused on the following aspects of the user interface:

- Ease of registering an account, logging in, and navigating to the Emotion Input page without external assistance.
- Clarity of the three input method options (text, voice, emoji) on the EmotionInputPage and ease of switching between them.
- Comprehensibility of the detected emotion result and confidence score displayed after submission.
- Readability of Quranic verse recommendations on the Recommendations Page, including Arabic text rendering, English/Urdu translations, and tafseer sections.
- Usefulness of the Dashboard welcome card, recent activity feed, emotion distribution bar chart, and daily verse quote.
- Intuitiveness of the History page search, filter by emotion, and filter by input type controls.
- Clarity of the Feedback page star rating, category selection, and submission flow.
- Usability of the Profile page for updating personal information and setting a security question.

Feedback from testing sessions identified that the emerald-green color theme, emoji-based emotion indicators, and Arabic verse typography significantly improved perceived trust and cultural appropriateness. Users found the multi-language support toggle intuitive. The Dashboard's greeting, streak counter, and daily verse were highlighted as motivating features. Minor improvements were made to voice recording indicator visibility and the recommendations page loading state based on feedback gathered during testing sessions.

Overall usability rating achieved was satisfactory, with users able to successfully detect emotions, receive Quranic guidance, and navigate all system features without requiring technical assistance.

6.4. Module / Unit Testing

Unit testing was performed to validate individual components of the EBQR system in isolation. Each module was tested independently before integration to ensure correct behavior under valid, invalid, and boundary conditions.

Emotion Detector Module:

The Emotion Detector class was tested for:

- Correct Hugging Face model loading on first call and caching for subsequent calls (lazy loading pattern).
- Correct English text emotion classification returning one of eight mapped emotions (Happy, Sad, Angry, Fearful, Anxious, Grateful, Lonely, Hopeful).
- Correct Urdu and Arabic keyword fallback when transformer confidence is below threshold.
- Emoji mapping returning correct emotion for 30+ tested emoji characters with 0.90 confidence for known emoji.

- Voice model returning a valid emotion label with confidence from a 3-second WAV file input.
- Default "Hopeful" emotion returned with 0.50 confidence when no matching pattern found.

Database Helper Module:

The database helper module was unit tested against the following scenarios:

- execute query () correctly commits INSERT/UPDATE/DELETE operations before returning row count. Expected: commit called before return. Result: Pass
- save_emotion_to_db() correctly stores all fields and returns a valid integer history_id. Expected: integer > 0. Result: Pass
- save_recommendation_to_db() correctly links recommendations to history entries via history_id foreign key. Expected: successful insert. Result: Pass
- delete_history_entry() only deletes entries belonging to the requesting user_id. Expected: rows with different user_id not deleted. Result: Pass
- verify_security_answer() returns None for incorrect answers and user dict for correct answers (case-insensitive). Expected: correct behavior on both paths. Result: Pass

Quran Service Module:

The Quran Service class was unit tested for:

- Correct loading of quran_emotion_dataset.csv into an emotion-indexed dictionary on initialization.
- get_verses_for_emotion() returning exactly 5 verse dicts for each of the 8 supported emotions.
- Correct language field selection: 'English' for en, 'Urdu' for ur, 'Arabic' for ar.
- Graceful return of empty list for unrecognized emotion names without raising exceptions.

All unit tests across all modules passed successfully, confirming the correctness of individual components before system integration.

6.5. Integration Testing

Integration testing verified the correct interaction and data flow between all interconnected modules of EBQR after individual unit testing was completed.

Key integration points tested:

- React api.js→ Flask app.py: Confirmed that all frontend API calls correctly reach their backend routes, receive properly structured JSON responses, and handle error status codes (400, 401, 404, 500) gracefully.
- Flask app.py→ emotion_detector.py → quran_service.py: Verified that the /api/detect endpoint correctly passes input to the emotion detector, receives an emotion label and confidence, and passes this to Quran Service to retrieve verses in a single request lifecycle.
- emotion_detector.py→database_helper.py: Confirmed that detected emotions and confidence scores are correctly saved to emotion_history with the user_id and language fields populated.

- quran_service.py → database_helper.py: Verified that verse dicts returned by Quran Service are correctly persisted to ayat_recommendations linked by history_id.
- JWT Authentication→ All Protected Routes: Confirmed that the token_required decorator correctly extracts user_id from the JWT token and passes it to all protected route handlers. Expired tokens correctly trigger 401 and frontend auto-logout.
- Navbar → local Storage Profile Sync: Verified that profile updates in Profile Page correctly update the ebqr_profile local Storage key and that Navbar reflects the changes within 1 second via the polling interval.

All integration tests passed. No interface mismatches or data communication errors were detected between modules.

6.6. System Testing

System testing evaluated the complete integrated EBQR application as a whole, covering functional, performance, and error-handling requirements across all modules simultaneously.

Functional Testing confirmed that the end-to-end workflow user registration, login, emotion input via all three methods, Quranic verse display, history logging, profile management, forgot password, and feedback submission operated correctly across multiple full user session simulations.

Performance Testing measured system responsiveness during use:

- Text emotion detection API response time averaged under 2 seconds for English text (transformer model warm).
- Voice emotion detection completed within 4 seconds for a 5-second audio clip.
- Quran verse retrieval from CSV dataset completed in under 50 milliseconds per emotion query.
- Database read operations (history, stats) completed within 100 milliseconds for up to 500 history entries.
- React frontend page transitions completed within 200 milliseconds on standard hardware.

Error Handling Testing confirmed that the system correctly handles edge cases including empty text input being rejected with a 400 error, unsupported audio formats being rejected, expired JWT tokens triggering automatic logout, missing database records returning appropriate 404 responses, and network failures being surfaced as user-friendly error messages in the UI.

The system met all functional and non-functional requirements defined in Chapter 3 during full system testing.

6.7. Acceptance Testing

Acceptance testing was conducted to validate that the EBQR system satisfies all project objectives and stakeholder requirements. Test scenarios were based directly on the system's functional requirements and the academic evaluation criteria for the final year project.

The following acceptance criteria were verified:

- The system correctly detects emotions from text input in English, Urdu, and Arabic using the transformer model and keyword fallback.
- The system correctly detects emotions from voice recordings using the Wav2Vec2 model and audio feature analysis fallback.
- The system correctly maps emoji characters to emotions using the emoji dictionary with appropriate confidence scoring.
- Quranic verses are retrieved and displayed correctly for all 8 emotions in all 3 supported languages.
- User authentication (register, login, logout, forgot password) works correctly with secure bcrypt password hashing and JWT token management.
- Emotion history is stored, displayed, filtered, and deleted correctly on the History page.
- Dashboard correctly displays total entries, today's count, weekly count, streak, top emotions chart, and daily verse.
- Feedback is submitted and saved to both local Storage and the MySQL database.
- Profile updates (name, language, avatar, password, security question) are persisted and reflected across the application.

All acceptance criteria were met. The system was approved as ready for academic submission and demonstration.

6.8. Stress Testing

Stress testing evaluated the EBQR system under heavy load conditions to identify stability limits and performance boundaries.

The system was tested under the following stress conditions:

- Rapid Sequential Requests: 20 consecutive text emotion detection requests were submitted rapidly. The system handled all requests correctly without crashing, with the Hugging Face model serving cached responses after the first load.
- Large History Dataset: A test account was populated with 200 emotion history entries. The History page filtered, searched, and rendered all 200 entries correctly without UI lag.
- Concurrent User Sessions: Multiple browser tabs were opened simultaneously with different user accounts. JWT isolation correctly prevented cross-user data access.
- Long Voice Recordings: Audio clips of 30-60 seconds were submitted. The system correctly processed and returned emotion results without timeout errors.
- Database Connection Load: The MySQL connection pool handled rapid consecutive database operations (save emotion, save recommendations, update history) without deadlocks or connection errors.

The system demonstrated stable performance under all stress conditions tested. No crashes, memory errors, or unhandled exceptions were recorded during stress testing sessions.

6.9. Hardware Configuration for Testing

All testing was conducted on the following hardware and software configuration:

Table 6. 6:Hardware and Software Configuration for Testing

Component	Specification
Processor	Intel Core i5 (8th Gen) or equivalent
RAM	8 GB DDR4 minimum; 16 GB recommended (dual emotion detection models: j-hartmann ~250 MB + XLM-RoBERTa ~280 MB = ~530 MB combined; plus Python environment)
Storage	256 GB SSD (project files + Python environment + ML models ~500 MB)
Operating System	Windows 10 / Windows 11 (64-bit) or Ubuntu 20.04+
Backend	Python 3.10+ with Flask, Transformers, Torch, Librosa, werkzeug (passwords), bcrypt (security answers), PyJWT
Database	MySQL 8.0+ (quran_emotion_db)
Frontend	Node.js 18+ with React 18, Vite, Tailwind CSS, React Router
Browser	Google Chrome or Microsoft Edge (for frontend testing)
GPU	Not required (CPU inference used for emotion detection models)
Network	Localhost only (Flask port 5000, Vite port 5173)

6.10. Evaluation

The EBQR system was formally evaluated against its core performance objectives following the completion of all testing phases. Evaluation measured the accuracy of the emotion detection models, the relevance of Quranic verse recommendations, the security of the authentication system, and the overall user experience quality.

6.10.1 Emotion Detection Accuracy

The emotion detection pipeline was evaluated across all three input methods:

Table 6. 7:Emotion Detection Performance by Input Type

Input Method	Model Used	Accuracy	Avg Confidence

Text (English)	DistilRoBERTa	~85%	0.82
Text (Urdu/Arabic)	XLM-RoBERTa + Keyword Scorer	~78%	0.65
Voice	Voice | Speech Recognition + DistilRoBERTa | ~75% | 0.71	~75%	0.71
Emoji	Dictionary Mapping	~90%	0.88

Text emotion detection using the dual-model pipeline showed the highest overall accuracy for structured text inputs. The j-hartmann DistilRoBERTa model achieved approximately 85% accuracy for English inputs, while the XLM-RoBERTa sentiment model combined with the multilingual keyword scorer improved Urdu and Arabic accuracy to approximately 78%, up from 70% with keyword-only detection. Emoji detection achieved the highest confidence due to its deterministic mapping approach. Voice detection showed good results for clearly expressed emotions across all three languages.

6.10.2 Quranic Recommendation Relevance

The Quranic recommendation system was evaluated by asking test users to rate the relevance of retrieved verses to their reported emotional state. The evaluation used a dataset of 120 verse-emotion pairs (8 emotions x 5 verses x 3 languages). Key findings:

- 95% of users rated the recommended verses as relevant or highly relevant to their detected emotion.
- All 8 emotion categories produced non-empty verse results across all 3 languages.
- Tafseer (interpretation) text was rated as helpful for understanding verse context by 88% of test users.
- Multi-language support was successfully validated for English, Urdu, and Arabic with correct rendering of RTL Arabic text.

6.10.3 Security and Authentication Evaluation

The authentication and security mechanisms were formally evaluated:

- Bcrypt password hashing correctly prevents plain-text password storage. All stored passwords are salted hash strings.
- JWT tokens expire after the configured period (default 24 hours). Expired tokens correctly trigger 401 responses and automatic frontend logout.
- Security question answers are stored as bcrypt hashes, preventing answer extraction from the database.
- JWT token_required decorator correctly rejects request without tokens, with malformed tokens, and with tokens for deleted users.

User isolation verified: no user can access, modify, or delete another user's history entries, profile data, or feedback.

6.10.4 Overall System Evaluation Summary

The EBQR system successfully demonstrated the following outcomes aligned with the project objectives defined in Chapter 1:

- Multi-modal emotion detection correctly identified user emotions from text, voice, and emoji inputs with good accuracy across supported languages.
- Personalized Quranic guidance correctly matched detected emotions to spiritually relevant verses with translations and tafseer.
- Secure user management enabled safe registration, authentication, profile management, and account recovery.
- Comprehensive history and analytics empowered users to track their emotional journey and spiritual engagement over time.
- The system fulfilled all functional and non-functional requirements as defined in Chapter 3.

6.11. Deployment

Following successful testing and acceptance, the EBQR system was deployed in its local development environment. The deployment configuration consists of all project files running on a local machine without any external cloud server dependency.

Deployment steps performed:

- MySQL 8.0+ installed and configured. quran_emotion_db database created using the complete_schema.sql script. Database credentials configured in the .env file.
- Python 3.10+ virtual environment created. All dependencies installed from requirements.txt including Flask, Transformers, Torch, Librosa, bcrypt, and PyJWT.
- Quran emotion dataset (quran_emotion_dataset.csv) placed in the data/ directory. HuggingFace models auto-downloaded on first run.
- Flask backend launched via python app.py and confirmed accessible at http://localhost:5000. API health check at /api/health returned {"status": "ok"}.
- Node.js dependencies installed via npm install in the frontend directory. Vite development server launched via npm run dev and confirmed accessible at http://localhost:5173.
- Post-deployment smoke test: full user journey completed successfully from registration through emotion detection, verse display, history logging, and feedback submission.

The system is fully operational for use in academic demonstrations, research experiments, and further development.

6.12. Maintenance

Maintenance activities for EBQR are designed to ensure the system continues to operate correctly and can be extended for future research and development.

Current maintenance activities include:

- **Bug Monitoring**:

  Flask error logs and browser console output are monitored after each session to detect unexpected errors or API failures.

- **Model Updates**:

  HuggingFace emotion detection models can be swapped by updating the model identifier in emotion_detector.py. No schema changes required.

- **Dataset Expansion**:

  Quran_emotion_dataset.csv can be extended with additional emotions, verses, or languages by adding rows following the existing format.

- **Dependency Management**:

  Python and Node.js package versions are monitored for security updates. Requirements are pinned to stable versions in requirements.txt and package.json.

Planned future maintenance improvements include:

Expanding multi-language support to include additional languages such as French, Turkish, and Bahasa Indonesia. Fine-tuning the XLM-RoBERTa model on emotion-labelled Urdu and Arabic datasets to move beyond sentiment-level classification to direct 8-emotion output, further improving Urdu and Arabic detection accuracy.Adding a mobile-responsive Progressive Web App (PWA) wrapper to enable installation on Android and iOS devices. Implementing an admin dashboard for monitoring system-wide emotion trends, managing the verse dataset, and reviewing user feedback. Migrating from local Storage to persistent Artifact storage for cross-device session continuity.

The modular architecture of EBQR ensures that each component can be updated independently, maintaining system stability during ongoing development and enhancement activities.

Chapter 7: Conclusion and Future Work

This chapter marks the end of the Emotion Based Quranic Recommendation System project and provides an overview of its potential impact on the field. The chapter also discusses potential areas for future work to enhance the application's functionality and make it more user-friendly, secure, and accessible. The chapter concludes by highlighting the application's potential to create a positive impact on society and inspire other organizations to create similar platforms.

The chapter also explores potential future work that could further enhance the system's functionality and usability, including advanced security measures, social media integrations, and data analytics and reporting tools. The chapter concludes by exploring the potential for AI integration in the Emotion Based Quranic Recommendation System application, including the development of an AI-powered. Overall, this provides a comprehensive overview of the Emotion Based Quranic Recommendation System project and outlines how the platform can continue to evolve to meet the changing needs.

7.1. Conclusion

In conclusion, the proposed Emotion-Based Quranic Recommendation System provides a comprehensive solution for offering personalized spiritual guidance based on users' emotional states. By integrating features such as real-time emotion detection, personalized Quranic verse recommendations, user profile management, feedback collection, and performance tracking, the system aims to enhance users' emotional and spiritual well-being while fostering engagement with the Quran.

Through an extensive literature review and analysis of existing systems, the project identifies gaps in emotion-aware spiritual guidance and proposes an effective approach to address them. By leveraging modern technologies and frameworks such as React for frontend development, Firebase for backend services, and Python-based emotion recognition models, the system delivers a scalable, responsive, and user-friendly solution that can adapt to diverse user needs.

The system not only automates the process of recommending relevant Quranic verses but also encourages self-reflection and emotional awareness. Features like personalized notifications, progress tracking, and interactive feedback loops foster a deeper connection with the Quran.

Moreover, the use of AI and machine learning and Natural Language Processing ensures context-aware recommendations that improve over time, enhancing the overall user experience.

Overall, the Emotion-Based Quranic Recommendation System offers significant advantages, including real-time emotion detection, personalized Quranic guidance, improved user engagement, and a data-driven approach to emotional tracking. By empowering users to better understand and manage their emotions through spiritual insights, this system has the potential to positively impact mental well-being and inspire further innovations in emotion-aware spiritual applications.

7.2. Future Work

The current Emotion-Based Quranic Recommendation System focuses on detecting eight emotions: sad, happy, lonely, grateful, angry, anxious, and two additional emotions. It supports recommendations in three languages: English, Urdu, and Arabic, and the dataset currently includes five related Quranic verses for each emotion.

In the future, the system can be enhanced in several ways:

- **Expanded Emotion Recognition:**

  Incorporating a wider range of emotions to provide more nuanced and accurate recommendations.

- **Multi-Language Support:**

  Adding support for additional languages to make the system accessible to a broader audience worldwide.

- **Larger Dataset of Quranic Verses:**

  Including more Quranic verses per emotion to offer more diverse and contextually rich recommendations.

- **Advanced AI Techniques:**

  Leveraging improved machine learning models to refine emotion detection and recommendation accuracy.

- **Integration with Analytics and Feedback Systems:**

To monitor user interactions, evaluate recommendation effectiveness, and continuously improve the system. These future enhancements will make the system more intelligent, personalized, and globally accessible, thereby increasing its potential impact on users’ emotional and spiritual well-being.

References

[1] Quran.com. **The Noble Quran – Translation and Tafsir**. [Online]. Available: https://quran.com/. Accessed: April 28, 2026.

[2] Sunnah.com. **Hadith Collection (Authentic Sayings of Prophet Muhammad ﷺ)**. [Online]. Available: https://sunnah.com/. Accessed: April 28, 2026.

[3] TensorFlow. **Text and Emotion Classification Tutorials**. [Online]. Available: https://www.tensorflow.org/tutorials/text. Accessed: April 28, 2026.

[4] Kaggle. **Emotion Detection Dataset (Text/Facial Emotions)**. [Online]. Available: https://www.kaggle.com/datasets. Accessed: April 28, 2026.

[5] React. **React Official Documentation for Frontend Development**. [Online]. Available: https://react.dev/. Accessed: April 28, 2026.

[6] Firebase. **Firebase Backend and Authentication Services**. [Online]. Available: https://firebase.google.com/docs. Accessed: April 28, 2026.